OPERATING SYSTEMS · TSINGHUA

操作系统原理

113 个视频的速学笔记

抽帧读图，把 22.7 小时的课程压成一份可扫读的骨架：
每节一句主旨 · 3–6 个关键点 · 一个考点 · 考前速览 40 条

清华大学 操作系统原理 · 113 个录屏 · 2023 版

目　录

01 先读这一页

02 第 0 章　实验课：环境准备与 Lab3

03 第 1 章　概述、启动与硬件基础

04 第 2 章　中断、异常与系统调用

05 第 3 章　内存管理基础

06 第 4 章　非连续内存分配

07 第 5 章　虚拟内存

08 第 6 章　页面置换算法

09 第 7 章　进程与线程

10 第 8 章　CPU 调度

11 第 9 章　同步与互斥

12 第 10 章　信号量、管程与经典同步问题

13 第 11 章　死锁与进程间通信

14 第 12 章　文件系统

15 附录 A：关于这批视频的几个提醒

16 附录 B：考前速览 40 条

先读这一页

这份笔记是怎么来的（以及它的局限）

视频本身没法「看」，我做的是**抽帧读图**：每个视频取 6—12 个时间点，把画面拼成 4 格一张的缩略图，逐张读出 PPT 上的标题、要点、公式和示意图，再写成笔记。

所以要清楚两件事：

- **内容以 PPT 为准**，老师口头讲的推导、举的例子、强调的考点，笔记里只保留了能从画面确认的部分。
- **代码画面基本读不出细节**（实验课和部分源码走查的视频），这类小节只概括了它在演示什么机制，没有逐行抄代码。

一句话：这份笔记适合**快速建立框架、复习时对照回忆**；真要抠细节，还得回到对应视频的那几分钟。

三种读法，按你的时间选

你有多少时间	怎么用
30 分钟	只看下面的「课程地图」+ 附录 B「考前速览 40 条」
3 小时	每章先读章导读，再扫该章各节的「主旨」一行，挑不懂的点开关键点
完整过一遍	按章顺序读，配合原视频：先看笔记定位，再跳到对应时间点看讲解

课程地图

章	主题	视频数	时长
0	实验课：环境准备与 Lab3	6	2.3 h
1	概述、启动与硬件基础	14	3.0 h
2	中断、异常与系统调用	5	2.0 h
3	内存管理基础	4	0.7 h
4	非连续内存分配	5	1.0 h
5	虚拟内存	5	1.3 h
6	页面置换算法	10	1.9 h
7	进程与线程	14	1.8 h
8	CPU 调度	6	1.2 h
9	同步与互斥	8	1.5 h
10	信号量、管程与经典同步问题	11	2.2 h
11	死锁与进程间通信	9	1.9 h
12	文件系统	16	1.9 h
—	**合计**	**113**	**22.7 h**

各章之间是什么关系

```
第 1 章  导论 + 硬件地基（中断/栈/特权级）
   └ 第 2 章  中断异常与系统调用  ← 全课的「入口」
         ├ 内存线：第 3 章 基础 → 第 4 章 分段分页 → 第 5 章 虚拟内存 → 第 6 章 置换算法
         └ 进程线：第 7 章 进程线程 → 第 8 章 调度
                                  └ 第 9 章 同步 → 第 10 章 信号量/管程 → 第 11 章 死锁/IPC
第 12 章  文件系统（相对独立，随时可看）
```

如果时间紧，**内存线和进程线二选一先打通**，不要两条线交替看——它们的问题意识完全不同。

第 0 章　实验课：环境准备与 Lab3

> 这批 6 个视频不是理论课，而是实验课（Lab0 环境准备 + Lab3 虚拟内存）。它们回答一个很实际的问题：这门课的知识怎么落到代码上。Lab3 的 do_pgfault 与 FIFO 换出，正好是第 5、6 章理论的一次完整实现，建议学完第 6 章后再回看。

0.0 清华 操作系统原理 Lab3.1（23.1 分钟）

- **主旨**：交代实验三虚拟内存管理的目标——在实验二的页表机制上实现 Page Fault（缺页异常）处理与页面置换，并给出两个必做练习、关键数据结构与页访问异常的处理路径。
- **关键点**：
  1. 实验三在实验二"物理内存连续分配 + 二级页表"的基础上，借助页表机制和实验一的中断异常处理机制完成 Page Fault（缺页异常）处理与 FIFO 页面置换，原理上的最大区别是"在磁盘上缓存内存页"，从而提供一个比实际物理内存空间"更大"的虚拟内存空间。
  2. 练习1 需在 LAB2 EXERCISE 1 处补全 do_pgfault(mm/vmm.c)，为未被映射的地址建立物理页映射；设置访问权限时要考虑所在 VMA 的权限，且必须操作内存控制结构所指定的页表，而不是内核页表，通过 check_pgfault() 测试会输出 "check_pgfault() succeeded!"。
  3. 练习2 需补全 vmm.c 的 do_pgfault 以及 swap_fifo.c 中的 map_swappable 与 swap_out_victim，实现基于 FIFO 的页面替换，在 LAB2 EXERCISE 2 处填写代码，通过 check_swap() 测试即基本正确；扩展练习是实现识别 dirty bit 的 extended clock 页替换算法（非必做，酌情加分）。
  4. 初始化顺序为 pmm_init()、pic_init()、idt_init()，随后由 kern_init 调用 vmm_init()（kern/mm/vmm.c），vmm_init 内部再调用 check_vmm()，由 check_vma_struct() 与 check_pgfault() 分别自检 VMA 结构与缺页处理。
  5. vma_struct 表示"使用同一页目录表的一段虚拟地址区间"，含 vm_mm、vm_start、vm_end、vm_flags 和按起始地址排序的 list_link；mm_struct 用 mmap_list 串起所有 VMA，用 mmap_cache 缓存最近访问的 VMA 以加速查找，pgdir 指向页目录表，sm_priv 存放 swap manager 的私有数据。
  6. 页访问异常路径为 vector14 → alltraps → trap/trap_dispatch → pgfault_handler → do_pgfault；Page-Fault Error Code 中 P 区分"页不存在"与"页级保护违规"，W/R 区分读/写，US 区分用户态/超级用户态，RSVD 表示保留位被置 1，ID 表示是否由取指引起。
- **易错/考点**：Error Code 各位含义（P=0 是缺页、P=1 是保护违规）以及 do_pgfault 必须使用 mm->pgdir 而非内核页表，是最易失分处。

0.0 清华 操作系统原理 Lab3.2（14.1 分钟）

- **主旨**：围绕页换入换出机制提出并回答五个设计问题：换出哪一页、虚拟页与磁盘扇区如何对应、何时换入换出、用什么数据结构支撑、以及具体怎么实现。
- **关键点**：

  1. 换出页的选择由页替换算法决定（kern/mm/swap.c），课程列出 FIFO（First In First Out，Lab3-2 必做）、Clock 与 Enhanced Clock 三种，实验用 check_swap() 验证。
  2. 虚拟页与磁盘扇区的对应关系复用 PTE：页不在内存时该表项被解释为 swap_entry_t，24 位 offset 记录磁盘上的换出位置，7 位 reserved 保留，最低位为 0 表明它指向的是 swap 项而非物理页。
  3. 页表项（Figure 4-5）中 P=1 表示页存在、R/W=1 可写、U/S=1 允许 ring 3 访问、A=1 表示已被访问、D=1 表示已被写过，这些标志正是替换算法挑选牺牲页的依据。
  4. 换入由缺页驱动：check_swap() 触发 page fault 后进入 do_pgfault() 把页从磁盘读回；换出分主动策略与被动策略，ucore 采用被动策略，即内存不足时才换出。
  5. struct Page 增加 pra_page_link 与 pra_vaddr 两个域，前者把可换出的物理页串成链表、后者记录该页对应的虚拟地址；swap_manager 抽象出 map_swappable（把页加入可换出集合）、swap_out_victim（挑出牺牲页并换出）、check_swap（自检）三个函数指针。
  6. swap_check() 的测试场景是：mm_create() + vma_create() 建立 4KB~24KB 的访问区间，free_page 只保留 4 个物理页却连续访问 5 个虚拟页，人为制造缺页，再由 check_content_set/check_content_access 与 _fifo_check_swap() 校验置换结果。

- **易错/考点**：页不在内存时 PTE 被复用为 swap_entry_t（最低位为 0），不要与存在位 P 混淆；FIFO 只按进入内存的先后淘汰，完全不看访问频度。

0.0 清华 操作系统原理 Lab3.3（31.4 分钟）

- **主旨**：以源码走查的方式串起从 kern_init 到 do_pgfault、再到 FIFO 换出的完整执行路径，指明练习代码应填在哪些函数里。
- **关键点**：

  1. kern_init() 依次调用 pmm_init()、pic_init()、idt_init()、vmm_init()、ide_init()、swap_init()、clock_init()，虚拟内存与换页子系统在时钟中断使能之前完成初始化。
  2. check_vma_struct() 先用 mm_create() 建 mm、用 vma_create(0, PTSIZE, VM_WRITE) 建 VMA 插入 mmap_list，再按 step 批量创建与查找 VMA，校验链表按 vm_start 有序且 find_vma 命中正确区间。
  3. check_pgfault() 建好 mm 与 VMA 后循环写 0~4 页触发缺页；do_pgfault 中先用 get_pte(mm->pgdir, addr, 1) 取页表项，若该表项为 0 则用 pgdir_alloc_page 分配物理页并建立映射。
  4. do_pgfault 的另一分支处理换入：PTE 存在但最低位为 0 时是 swap 项，调用 swap_in(mm, addr, &page) 从磁盘读回，再用 page_insert 更新页表，并用 swap_map_swappable 把该页设为可换

出。

5. swap_fifo.c 中 fifo_init 初始化 free_list，fifo_map_swappable 用 list_add 把页的 pra_page_link 挂到队尾，fifo_swap_out_victim 从队首取第一个页作为牺牲页并 list_del，体现"先进先出"。
6. 换出时先判断页是否被写过，脏页需用 swapfs_write 写回磁盘，之后把 PTE 改写为 swap_entry_t 并调用 tlb_invalidate 刷新 TLB（快表）。

- **易错/考点**：画面以代码/终端为主；核心是 do_pgfault 的"分配新页"与"从 swap 换入"两条分支，以及 FIFO 用 list_add 入队尾、list_del 取队首的配对关系。

0.1 实验准备1（11.7 分钟）

- **主旨**：Lab0 实验准备开篇，说明课程实验的环境构成、工具链与 x86-32 硬件基础，并演示如何下载、导入 VirtualBox 实验镜像。
- **关键点**：

1. 实验在预装好的 Ubuntu 虚拟机上完成，工具分为 shell 命令（ls、cd、rm、pwd 等）、系统维护工具（apt、git）、源码阅读与编辑工具（understand、gedit、vim）、源码比较工具（diff、meld）、开发编译调试工具（gcc、gdb、make）以及硬件模拟器 qemu。
2. x86-32 硬件基础包括 Intel 80386 的运行模式概述、内存架构概述、寄存器概述三部分，此外还需了解 ucore 的面向对象编程方法与通用数据结构。
3. 虚拟硬盘压缩包放在网盘（pan.baidu.com/s/1gd6PM6J），解压后得到 mooc-os.vbox、mooc-os.vbox-prev、mooc-os.vdi 三个文件，合计约 4GB 多，用 VirtualBox 打开 .vbox 即可加载。
4. 虚拟机登录用户名是 moocos，口令是"空格键"。
5. 在 VirtualBox"新建虚拟机"向导中为 mooc-os 分配内存，推荐 512MB，界面可调范围是 4MB 到 2048MB。

- **易错/考点**：口令是空格键而不是空字符串，且导入时应通过 .vbox 配置文件加载，不要直接打开 .vdi。

0.2 实验准备2（23.7 分钟）

- **主旨**：在虚拟机里配好共享文件夹，并用真实命令演示 shell 操作、依赖安装、源码检索、git 提交、make 编译与 gdb 调试的完整实验工作流。
- **关键点**：

1. 在 VirtualBox 的"设置→共享文件夹"中把宿主机 /tmp 挂载进虚拟机，名称 /tmp、自动挂载"是"、访问权限"固定分配"，用于宿主机与 guest 之间传文件。
2. shell 命令 ls、cd、rm、pwd、mkdir、find 基于 bash（Bourne-Again SHell），完成对文件与目录的基本操作。
3. prepare.sh 用 apt-get 一次性装齐实验依赖：build-essential、git、qemu-system-x86、vim-gnome、gdb、make、diffutils、exuberant-ctags、tmux、openssh-server、cscope、meld，执

行前先 sudo apt-get update。

4. 源码定位靠 find ./labcodes/ -name "*.c" 一类组合（如再接 | less）快速找到 kern/init/init.c、kern/mm/vmm.c 等目标文件。
5. git 流程为：修改 README 后 git status 显示 "Changes not staged for commit"，git add README 进入暂存区，git commit -m "update README" 提交后工作区变 clean。
6. gcc 是 C 语言编译器、gdb 是执行程序调试器、make 是软件工程管理工具且必须依赖 makefile 才知道如何编译链接；执行 make qemu 后 ucore 打印 "(THU.CST) os is loading ..." 与 "Kernel executable memory footprint: 64KB"。

- **易错/考点**：make 离开 makefile 无法工作，qemu 只是硬件模拟器，需与 gdb 配合才能做内核源码级调试。

0.3 实验准备（31.6 分钟）

- **主旨**：补齐 x86-32 的运行模式、地址空间与寄存器基础，说明 ucore 如何用 C 语言模拟面向对象与通用双向链表，最后演示 Eclipse 配合 gdb 的源码级调试。
- **关键点**：

1. 80386 有实模式、保护模式、SMM 模式和虚拟 8086 模式四种运行模式；加电后处于实模式，可访问的物理内存不能超过 1MB，无法发挥 32 位 CPU 的 4GB 内存管理能力。
2. 保护模式支持内存分页机制从而良好支持虚拟内存，并支持多任务与 0~3 共 4 个优先级：操作系统运行在最高优先级 0，应用程序运行在较低级别，配合检查机制既能在任务间安全共享数据也能很好地隔离任务。
3. 地址是访问内存空间的索引；80386 是 32 位处理器，可寻址物理内存为 2^32=4GB，物理地址是处理器提交到总线上访问内存和外设的最终地址、一个计算机系统只有一个物理地址空间，而线性地址空间是各应用程序在虚存管理下看到的、相互隔离的地址空间。
4. 80386 寄存器分 8 组：通用寄存器、段寄存器、指令指针寄存器、标志寄存器、控制寄存器、系统地址寄存器、调试寄存器、测试寄存器。
5. EIP 的低 16 位就是 8086 的 IP，存放下一条要执行指令的地址，在分段地址转换中表示段内偏移；EFLAGS 的 IF（中断允许标志）由 CLI/STI 控制，只对可屏蔽外部中断有效，对不可屏蔽中断和故障中断没有作用。
6. ucore 用 C 语言模拟面向对象：pmm_manager 把 init、init_memmap、alloc_pages、free_pages、nr_free_pages、check 组织成函数指针表；通用双向链表 list_entry 只有 prev/next 两个指针，再靠 le2page 与 to_struct（(char *)(ptr) - offsetof(type, member)）从链表节点反推出宿主 Page 结构的首地址；Eclipse 中在 kern_init 下断点后可单步进入 cons_init 观察执行。

- **易错/考点**：to_struct 是用节点地址减去成员偏移反推宿主地址、le2page 是它的特化用法；IF 位对 NMI（不可屏蔽中断）无效是常考细节。

第 1 章　概述、启动与硬件基础

> 本章一半是导论（OS 是什么、为什么学、历史与结构），一半是硬核铺垫（bootloader 启动链、C 函数调用的栈帧、内联汇编、x86 中断处理）。后者看着像实验课，其实是后面所有章节的地基——不懂中断怎么进入、栈怎么切换，第 2 章之后会寸步难行。

1.1 bootloader启动ucore os（19.8 分钟）

- **主旨**：串起 x86 从上电第一条指令到跳入 ucore 内核的完整启动链：BIOS → MBR → bootloader → ELF 格式 kernel。
- **关键点**：
  1. 加电后 CPU 处于实模式，CS = F000H、EIP = 0000FFF0H，实际地址 = Base + EIP = FFFFFFF0H，即 BIOS 所在的 EPROM（Erasable Programmable Read Only Memory）；此处第一条指令是一条长跳转指令，执行后 CS 与 EIP 都被更新，地址转换规则随即开始生效。
  2. 实模式下由 16 位段选择子（Segment Selector，如 CS、DS、SS）和 16 位偏移量（Offset）合成 20 位线性地址，即 Base + Offset。
  3. BIOS 把启动设备（软盘、硬盘、光盘、USB 盘）的第一个扇区——主引导扇区 MBR（Master Boot Record）的 512 字节——加载到内存 0x7c00，然后跳转到 0x7c00 处的第一条指令执行。
  4. bootasm.S 中 bootloader 先开 A20、置 CR0 的 PE 位使能保护模式，用 SEG_NULLASM / SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff) 等构造 GDT（null、code、data 三个描述符），长跳转刷新 CS 进入 32 位保护模式，再装入 PROT_MODE_DSEG 到 ds/es/fs/gs/ss、设 esp = start，最后 `call bootmain`。
  5. bootmain（C 语言）先从磁盘读入 ELF 头并校验 ELF_MAGIC，再按 program header 逐段把 p_offset/p_filesz 的内容读到 p_paddr，最后以 `((void (*)(void))ELFHDR->e_entry)()` 跳转到内核入口。
  6. 保护模式下段选择子不再直接给基址，而是作为 GDT/LDT 的索引取出段基址，加偏移得线性地址，再经分页机制（页目录/页表）转为物理地址。
- **易错/考点**：实模式段基址 = 选择子 << 4，保护模式选择子只是描述符索引；MBR 固定装入 0x7c00 是最常被问的数字。

1.1 课程概述（6.2 分钟）

- **主旨**：交代本课程的内容板块、实验体系与预备知识要求。
- **关键点**：
  1. 课程内容分三块：基本概念及原理、操作系统介绍、中断及系统调用。

2. 每讲的组织形式为"课程简介 — 练习 — 延伸"，练习环节在 uCore 操作系统上做实验，代码地址 https://github.com/chyyuu/mooc_os_lab，延伸环节讨论相关话题或故事。
3. 实验共 9 个：实验 0 准备、实验 1 系统启动及中断、实验 2 物理内存管理、实验 3 虚拟内存管理、实验 4 内核线程管理、实验 5 用户进程管理、实验 6 CPU 调度、实验 7 同步与互斥、实验 8 文件系统。
4. 预备知识要求：计算机组成原理（Intel 80386+）、数据结构、C 和汇编语言。

- **易错/考点**：—

1.2 c函数调用的实现（7.1 分钟）

- **主旨**：用汇编看清 C 函数调用时栈帧如何建立、现场如何保存与恢复。
- **关键点**：

1. 调用者先用 `pushal` 以及 `pushl %eax / %ebx / %ecx` 保存通用寄存器现场，再 `call foo`；返回后按 `popl %ecx / %ebx / %eax`、`popal` 逆序恢复。
2. `call` 自动把返回地址压栈；进入被调函数后 `pushl %ebp` 保存调用者帧基址、`movl %esp,%ebp` 建立新栈帧，EBP 指向"Caller's EBP"。
3. 栈帧从高地址到低地址依次是：参数（Argument 01、Argument 02...）、返回地址、Caller's EBP，其下才是局部变量；EBP 作帧基址不动，ESP 随压栈向低地址移动。
4. 函数返回时 `popl %ebp` 恢复调用者的帧基址，`ret` 弹出返回地址回到调用点，栈帧随之销毁。

- **易错/考点**：栈向低地址方向增长，所以图中"Low"在上、"High"在下；EBP 恒定指向帧底、ESP 动态变化。

1.2 什么是操作系统（16.1 分钟）

- **主旨**：从"资源管理者"与"硬件抽象提供者"两个角度定义 OS，并给出 Kernel 的组成与四大特征。
- **关键点**：

1. OS 首先是资源管理器：负责管理外设、分配资源（画面 2:00 处要点为"资源管理 / 管理外设、分..."）。
2. OS 位于应用软件之下、硬件之上，为应用软件提供服务支撑；办公软件、视频播放软件等都运行在 OS 之上（对应 Software—Application—System—Utility/Operating system—Shell/Kernel 的分层图）。
3. OS 把底层硬件抽象成上层可见的对象：CPU 抽象为进程、磁盘抽象为文件、内存抽象为地址空间。
4. Kernel 内部组件包括 CPU 调度器、物理内存管理、虚拟内存管理、文件系统管理、中断处理与设备驱动，uCore 按硬件层—内核（含系统调用接口、进程/线程管理、内存管理、文件系统）—库—应用的层次组织。

5. Kernel 特征之一是并发：计算机系统中同时存在多个运行的程序，需要 OS 管理和调度。
6. 另外三个特征是共享（"同时"访问与互斥共享）、虚拟（利用多道程序设计技术，让每个用户都觉得有一台计算机专门为他服务）、异步（程序执行不是一贯到底而是走走停停、推进速度不可预知，但只要运行环境相同，OS 需保证运行结果也相同）。

- **易错/考点**：并发（concurrency）不等于并行（parallelism）；异步的关键落点是"环境相同则结果必须相同"。

1.3 GCC内联汇编（12.9 分钟）

- **主旨**：掌握 GCC 内联汇编（inline assembly）的模板语法与约束符，从而能在 C 代码里直接插入特殊指令、发起系统调用。
- **关键点**：

1. 内联汇编是 GCC 对 C 语言的扩展，可直接在 C 语句中插入汇编指令；用途是调用 C 语言不支持的指令（如特权指令）或在 C 语言中手工优化。
2. 工作机制：用给定的模板和约束来生成汇编指令，在 C 函数内形成汇编源码。
3. 例 3 用 `__asm__ volatile ("int $0x80" : "=a"(_res) : "a"(11),"b"(arg1),"c"(arg2),"d"(arg3),"S"(arg4))` 发起系统调用：11 作为系统调用号放入 eax，四个参数分别经 ebx、ecx、edx、esi 传入。
4. 约束符含义：a = %eax、b = %ebx、c = %ecx、d = %edx、S = %esi、D = %edi，0 表示"与第一个操作数使用同一寄存器"。
5. 对应的编译结果为 movl $11,%eax；movl -28(%ebp),%ebx；movl -24(%ebp),%ecx；movl -20(%ebp),%edx；movl -16(%ebp),%esi；int $0x80；movl %eax,-12(%ebp)——返回值经 eax 写回 _res。

- **易错/考点**：画面以代码/终端为主；注意 `"=a"` 是输出约束、`"0"` 是复用第 0 个操作数寄存器的匹配约束。

1.3 为什么学习操作系统（16.4 分钟）

- **主旨**：说明 OS 课在整个计算机课程体系中的位置，以及学习它的工程价值与现实需求。
- **关键点**：

1. 这是一门综合课程，结合程序设计语言、数据结构、算法、计算机体系结构等多门不同的课程。
2. 尽管已经有 Windows、Linux 等很多成熟的操作系统，但操作系统仍在不停发展，在工控等各种领域有强烈需求。
3. 课程立论是操作系统很酷、很有用、有挑战。
4. 操作系统代码直接管理原始硬件，因而必须应对时间依赖行为、非法行为和硬件故障这三类问题。

5. 硬件层面要求良好的硬件管理与合理的资源分配，例如磁盘速度跟 CPU、内存相差很多，操作系统需要协调处理这种速度差异。

- **易错/考点**：—

1.4 X86中的中断处理（14.6 分钟）

- **主旨**：讲清 x86 中断/异常如何经 IDT 定位到中断服务例程（ISR），以及特权级切换时的栈变化、从中断返回与系统调用的实现路径。
- **关键点**：

1. 每个中断或异常都与一个中断服务例程（ISR，Interrupt Service Routine）关联，这种关联关系存放在中断描述符表（IDT，Interrupt Descriptor Table）中。
2. IDT 的起始地址和大小保存在中断描述符表寄存器 IDTR 中；IDT 表项为 8 字节的门描述符，中断门（Interrupt Gate）与陷阱门（Trap Gate）均含 Segment Selector、Offset 31..16、Offset 15..0，以及 DPL、P、Type 字段。
3. 中断发生时 CPU 用中断号索引 IDT，取出段选择子与偏移，经 GDT/LDT 得到段基址，与 Offset 拼成 ISR 入口的线性地址，再查页表得物理地址，随后压栈并跳转执行（Figure 6-8 Interrupt Procedure Call）。
4. 不涉及特权级变化时，硬件只压入 EFLAGS、CS、EIP；一旦发生特权级改变（Privilege-Level Change），还要切换到内核栈并额外压入 SS、ESP（若有错误码 Error Code 也一并压入），即从 Stack 1 换到 Stack 2。
5. 返回用 `iret`：它弹出 EFLAGS 和 SS/ESP（是否弹 SS/ESP 取决于是否改变特权级），而 `ret` 只弹出 EIP、`retf` 弹出 CS 和 EIP。
6. 用户程序通过系统调用访问 OS 内核服务，实现手段包括使用 Trap、用软件产生中断，或使用专用指令（SYSENTER/SYSEXIT）；kern_init 中遍历 vectors 表，以 `SETGATE(idt[i], 0, KERNEL_CS, __vectors[i], DPL_KERNEL)` 初始化 IDT，并用 `lidt` 装入 IDTR。

- **易错/考点**：iret 与 ret/retf 的弹栈差异、以及特权级变化时多压 SS/ESP 是高频考点；后半段画面以代码/终端为主（vectors.S、trap.c、Makefile）。

1.4 如何学习操作系统（1.9 分钟）

- **主旨**：给出本课程的学习方法立场——操作系统必须靠亲手实践来掌握，而非靠听讲和看书。
- **关键点**：

1. 画面板书写出学习效果的三级递进：「我听到的我会忘记」「我看到的我能记住」「只有我做过的我才能理解」。
2. 三者按感官参与度递增排列，把「做」置于「听」「看」之上，说明被动接收的留存率最低。
3. 由此推出本节的主张：理解操作系统原理要靠自己动手完成实现与调试，而不是记住结论。

- **易错/考点**：本节为学习方法引导，画面只有三行方法论文字，无技术考点。

1.5 完成过程1（29.1 分钟）

- **主旨**：通过实验一 bootloader 的编译、链接与调试，走通「BIOS 加电 → 加载主引导扇区 → 切换保护模式 → 跳入 C 代码」的完整启动链条。
- **关键点**：

  1. 实验目标：另写一个 bootloader，完成切换到 x86 保护模式并显示信息，为加载 ucore OS 做准备；整个 bootloader 执行代码必须小于 512 字节，才能放进硬盘主引导扇区。
  2. 构建链路为 gcc 编译 bootasm.S 与 bootmain.c 得到 bootblock.o，ld 以 `-Ttext 0x7c00` 链接，objcopy 转成二进制，终端打印「build 512 bytes boot sector: bin/bootblock 'success!'」。
  3. 关键编译选项：`-m32` 生成 x86 代码、`-gstabs` 生成 stabs 调试信息、`-ggdb` 生成 gdb 调试信息、`-nostdinc` 不搜索标准头文件路径、`-fno-stack-protector` 不生成缓冲区溢出检测代码、`-Os` 减小代码体积。
  4. bootasm.S 先 `cli` 关中断并把各段寄存器清零、建立栈，再通过 8042 键盘控制器端口 0x64/0x60 打开 A20（先等控制器不忙，再发 0xD1 写输出端口指令，再发 0xDF）。
  5. 保护模式切换顺序：装载 GDT（三个描述符：空、代码段 `STA_X|STA_R` 基址 0 限长 0xffffffff、数据段 `STA_W`）→ 置 `CR0.PE` → `ljmp $PROT_MODE_CSEG, $protcseg` 进入 32 位代码段；GDT 用「基址 0、限长 4GB」的平坦模型，使虚拟地址等于物理地址，切换过程中内存映射不变。
  6. bootmain.c 先从磁盘读入 ELF 头，校验 `e_magic == ELF_MAGIC`，再按 program header 逐个 `readseg` 把段加载到 `p_pa`，最后跳转到 `e_entry`；读盘走 PIO，经 0x1F2–0x1F7 端口设置扇区号与读命令。
  7. 调试方法：qemu 以 `-S -s` 启动后在 gdb 中 `target remote localhost:1234`、`break *0x7c00` 单步跟踪 BIOS 之后的引导代码；用 `-d in_asm -D bin/q.log` 导出执行的汇编指令，与 bootasm.S / bootblock.asm 对照。

- **易错/考点**：bootloader 的 512 字节上限、链接地址 0x7c00、`CR0.PE` 置位与 `ljmp` 远跳转的先后顺序是常考点；画面以代码/终端为主。

1.5 操作系统实例（5.5 分钟）

- **主旨**：用 UNIX、Linux、Windows 三大家族的谱系图，说明现实中的操作系统如何分化与衍生。
- **关键点**：

  1. UNIX 家族以贝尔实验室的 UNIX 为根，向上分出 UNIX BSD（伯克利软件发行版）与 System V 两大分支。
  2. 两大分支继续衍生出 Solaris、AIX/HP-UX、macOS 与 iOS 等商业系统，图中以厂商 logo 标注各支系归属。

3. Linux 家族以 Linux 内核（企鹅标志）为根，各发行版 Red Hat、Fedora、Ubuntu、SUSE、Debian、Gentoo 在其上组合不同软件栈。
4. Android 挂在 Linux 家族的分支上，说明移动操作系统复用了 Linux 内核。
5. Windows 家族以 PC 操作系统为主并逐渐向其他终端拓展（画面为微软各代产品与设备矩阵）。

- **易错/考点**：BSD 是 UNIX 的一个分支而非独立体系，Android 属 Linux 家族而非 UNIX 家族，谱系归属易混。

1.6 完成过程2（26.9 分钟）

- **主旨**：以 lab1 的调试函数与 lab2 的中断框架为例，讲清「栈回溯」与「中断/异常从入口到分派再返回」的实现路径。
- **关键点**：

1. `print_stackframe` 先用 `read_ebp()`、`read_eip()` 取当前帧，再循环打印 ebp、eip 以及 `(uint32_t*)ebp + 2` 起的 4 个参数，然后以 `eip = ((uint32_t*)ebp)[1]`、`ebp = ((uint32_t*)ebp)[0]` 逐层上溯。
2. 回溯终止条件为 `ebp == 0` 或层数达到 `STACKFRAME_DEPTH`，每层还调用 `print_debuginfo(eip-1)` 打印函数名与行号。
3. IDT 表项由 `struct gatedesc` 的位域描述（gd_off_15_0、gd_ss、gd_type、gd_dpl、gd_p、gd_off_31_16），由 `SETGATE(gate, istrap, sel, off, dpl)` 宏按偏移高低位、段选择子、门类型与特权级填充。
4. 中断入口 `__alltraps` 依次压入 ds/es/fs/gs 与 `pushal` 保存通用寄存器，构造出 trapframe，再以 `%esp` 作为参数 `call trap`，返回后 `trapret` 恢复寄存器、`addl $0x8, %esp` 丢弃中断号与错误码、`iret` 返回。
5. `trap_dispatch` 按 `tf->tf_trapno` 分派：`T_PGFLT` 交给 `pgfault_handler`、`T_SYSCALL` 交给 `syscall`，其余走默认分支打印 trapframe。
6. 扩展练习用 `T_SWITCH_TOU`（120）与 `T_SWITCH_TOK`（121）两个中断配合 `switch_to_user()` / `switch_to_kernel()`，在 `kern_init` 调用的 `switch_test()` 中来回切换内核态与用户态。
7. `kern_init` 建立 GDT 与 TSS：`ts.ts_esp0` 指向 `bootstack + KSTACKSIZE` 作为内核栈顶，`ltr(GD_TSS)` 装载任务寄存器，从而在用户态陷入内核时自动换栈。

- **易错/考点**：ebp 链的取值顺序（`[ebp]` 是新 ebp、`[ebp+4]` 是新 eip）、trapframe 压栈顺序与 iret 前清 8 字节错误码最易写错；画面以代码/终端为主。

1.6 操作系统的历史（11.4 分钟）

- **主旨**：从「纸带加载」到「多道并发」再到「普适计算」，解释操作系统为何演化成今天的样子。

- **关键点**：

  1. 早期计算机用纸带传输程序和数据，操作系统只起到加载程序的作用。
  2. CPU 等硬件速度快速提升后，单道执行使性能得不到充分利用，且内存容量越来越大，可以同时容纳多个程序。
  3. 单道方式下程序发出 I/O 请求后走 `startIO` → `waitIO` → `endIO`，CPU 在等待 I/O 期间空转（画面时序图中 read 与 I/O 设备严格串行）。
  4. 多道程序方式下，一个程序 `waitIO` 时操作系统 `schedule` 切换到另一个程序，I/O 完成中断到来后再切回，使 CPU 计算与 I/O 重叠。
  5. 引入 `timer interrupt` 后由时钟中断周期性触发 `schedule`，形成抢占式分时，避免单个程序长期霸占 CPU。
  6. 个人电脑操作系统的目标从「提高利用率」转为「用户体验与 API」：单用户、很多大型机服务和功能不再需要，但多应用协调与安全（电子商务、医疗记录）问题突出。
  7. 演变路径为：主机计算（Mainframe computing）→ 个人计算（Personal computing）→ 网络计算（Internet computing）→ 普适计算（Pervasive computing，含大数据收集、云计算与个性化服务）→ Future OS。

- **易错/考点**：多道程序靠 I/O 中断切换、分时靠 timer interrupt 抢占，两种切换触发源易混。

1.7 操作系统结构（10.3 分钟）

- **主旨**：比较单体/分层内核、微内核、外核与虚拟机监视器四种组织方式，说明「哪些功能该留在内核」的不同答案。
- **关键点**：

  1. 简单操作系统以 MS-DOS（1981 年起）为例，结构简单、缺乏模块划分与保护（画面右侧文字被截断，仅见「MS-DOS(1981~19...」）。
  2. Unix 操作系统于 1972 年由 Kenneth Thompson 和 Dennis Ritchie 在贝尔实验室设计。
  3. 分层视图：用户之上是 shells/命令、编译器和解释器、系统库，经 system-call interface 进入内核；内核内依次为信号与终端处理、字符 I/O 与终端驱动、文件系统与块 I/O、CPU 调度与页替换/请求调页/虚拟内存，最底层经内核接口对接终端控制器、设备控制器与内存控制器。
  4. 该分层图被标注为「用于实验的 uCore 操作系统」的结构参照。
  5. 微内核的设计思想是尽可能把内核功能移到用户空间，内核只保留 IPC 等基本机制，内存管理、文件系统、I/O、设备驱动、网络栈都以用户态服务进程形式挂在外围。
  6. 外核（exokernel）是更极端的架构：内核只做硬件资源的安全复用与保护，POSIX、TCP、VM、DFS、IPC 等抽象交给用户态的 library operating systems。
  7. VMM（虚拟机监视器）把单独的机器接口转换成很多幻象，每个虚拟机是一个原始计算机系统的有效副本并能完成所有处理器指令，从而让多个 Guest OS 共享同一套物理硬件。

- **易错/考点**：微内核（功能移到用户态）、外核（只复用硬件、抽象交给库 OS）、VMM（虚拟出多台机器）三者职责边界易混；3:51 处画面全黑（画面不清晰）。

1.8 小结（1.5 分钟）

- **主旨**：回顾本章的知识地图，把「是什么、为什么学、怎么学」与「实例、演变、结构」串成一条主线。
- **关键点**：
  1. 小结逐条列出本章目录：课程概述、什么是操作系统？、为什么学习操作系统？、如何学习操作系统？。
  2. 随后追加三条：操作系统实例、操作系统的演变、操作系统结构。
  3. 七条目录即本章的组织顺序，前四条回答概念与动机，后三条给出实例、历史与结构三个视角。
- **易错/考点**：本节仅回顾章节目录，未展开新知识点。

第 2 章　中断、异常与系统调用

> 核心问题只有一个：用户程序怎么安全地「求」内核办事。答案靠硬件撑起来——特权级（ring 0/3）、段描述符、TSS、IDT、MMU，一层层把「越权」变成「陷入内核」。这一章是全课最「硬」的一章，值得慢看。

2.1 X86特权级（28.7 分钟）

- **主旨**：说明 x86 如何用段选择子、段描述符与 TSS 在硬件层面隔离内核与用户程序，并支撑特权级之间的切换。
- **关键点**：
  1. 段选择子（segment selector）由 13 位 Index、1 位 TI（0 表示 GDT、1 表示 LDT）和 2 位 RPL（Requested Privilege Level，请求特权级）组成。
  2. 特权级检查由段描述符中的 DPL 与 CPL、RPL 共同完成，三者比较后不合规则拒绝访问并触发异常；数值越小特权越高，因此 ring 0 是内核、ring 3 是用户。
  3. 通过中断门（interrupt gate）进入内核时，门描述符给出目标代码段选择子和偏移，CPU 由中断向量定位到门描述符后转入处理程序。
  4. 从 ring 3 切到 ring 0 时 CPU 依 TSS 取出 SS0/ESP0 切换到内核栈，并把用户栈的 SS:ESP、EFLAGS、CS、EIP 以及 Error Code 依次压入内核栈，形成带特权级变化的中断栈帧。
  5. TSS（Task-State Segment，任务状态段）保存各级栈指针与寄存器现场，是硬件找到内核栈的唯一依据，其 32 位格式即 Figure 7-10 的 Task-State Segment。
  6. 本节目标为三件事：了解不同特权级的差别、了解当前 CPU 处在哪个特权级、了解特权级如何切换。
- **易错/考点**：CPL 取自当前执行的代码段、RPL 来自选择子本身，二者与 DPL 的比较方向（特权级数值不得低于目标段 DPL）是经典考点。

2.1 操作系统的启动（19.3 分钟）

- **主旨**：讲清机器从上电到把 OS 装入内存并交出控制权的完整顺序，同时界定中断、异常、系统调用三者的区别。
- **关键点**：
  1. DISK 存放 OS，BIOS 是基本 I/O 处理系统；上电后先做 POST（加电自检），寻找显卡并执行 BIOS。
  2. BIOS 把磁盘引导扇区中的 bootloader（512 字节）加载到内存 0x7c00，然后跳转 CS:IP = 0x0000:7c00。

3. bootloader 进一步把操作系统的代码和数据从硬盘加载到内存，再跳转到 OS 所在的内存地址，控制权移交给内核。
4. 三者按来源区分：系统调用源于应用程序主动请求服务；异常源于不良应用程序，指非法指令或其他坏的处理状态（如内存出错）；中断源于外设，如硬件设备的计时器和网络。
5. 处理时间上中断是异步的、异常是同步的、系统调用可异步也可同步；响应上中断对用户程序透明且可持续，异常会杀死程序或重新执行出错指令，系统调用表现为等待和持续。
6. 内核是被信任的第三方、是唯一能执行特权指令的部分，系统调用接口存在的意义就是方便应用程序。

- **易错/考点**：来源、同步/异步、响应方式三张对照表极易混淆，尤其"异常同步、中断异步、系统调用两者皆可"。

2.2 x86内存管理单元MMU（24.8 分钟）

- **主旨**：说明 x86 段机制与页机制如何把逻辑地址逐级翻译为物理地址，以及 uCore 如何建立 GDT 与页表。
- **关键点**：

1. 段寄存器可见部分只有选择子，隐藏部分缓存 Base、Limit、Access；隐藏的基址一直保留到选择子改变时才按新段表项更新。
2. 段机制用"段基址 + 段内偏移"得到线性地址，页机制再把 32 位线性地址切成 10 位页目录索引、10 位页表索引、12 位页内偏移。
3. 两级页表结构为 CR3 → 页目录（page directory）→ PDE → 页表（page table）→ PTE → 物理页框，PTE 中的物理页框号拼接页内偏移得到物理地址（Figure 4-2）。
4. 页表项关键标志位：R/W=1 该页可写，U/S=1 该页允许 ring 3 访问，A=1 该页被访问过，D=1 该页被写过。
5. 建立页表的步骤：分配一页作页目录并清零，把虚拟地址 0x00000000–0x7fffffff 映射到物理地址 0x00000000–0x3fffffff，再把低端 1MB 做恒等映射，最后设置 CR3 并置 CR0 的 bit31 开启分页。
6. uCore 内存管理初始化的四个阶段：bootloader（CS base=0，PG=no，虚拟地址基址 0x0）→ kernel 汇编阶段（CS base=0xC0000000，PG=no）→ 刚开启分页（CS base=0xC0000000，PG=yes）→ 全部完成（CS base=0，PG=yes，虚拟地址基址 0x0）。

- **易错/考点**：顺序不能颠倒——先段机制得线性地址、再页机制得物理地址；CR3→页目录→页表→物理页的层级与 PDE/PTE 的作用必须记牢。

2.2 操作系统的终端、异常、和系统调用（17.4 分钟）

- **主旨**：讲清中断与异常在硬件、软件两侧的处理流程，以及系统调用如何借助 API 和系统调用号实现。
- **关键点**：

1. 处理机制分硬件与软件两半：硬件负责设置中断标记（CPU 初始化时登记内部/外部事件的中断标记与中断事件 ID），软件负责保存当前处理状态。
2. 异常处理的四步是取得异常编号、保存现场、异常处理、恢复现场，其中异常处理要么杀死产生异常的程序，要么重新执行异常指令。
3. 系统调用由应用程序通过 API 间接触发：应用程序调用 printf() 时，实际由 C 库触发系统调用 write()，从用户态陷入内核态的系统调用处理程序。
4. 三套典型 API 体系：Win32 API 用于 Windows，POSIX API 用于 POSIX-based systems（包括 UNIX、LINUX、Mac OS X 的所有版本），Java API 用于 Java 虚拟机（JVM）。
5. 系统调用接口按系统调用号维护表的索引，据此调用内核态中预期的服务例程，并返回调用状态与返回值；用户只需拿到 API 并了解返回结果，实现细节被库隐藏。
6. 跨越操作系统边界的开销大于普通过程调用，来源包括建立中断/异常/系统调用号与服务例程映射的初始化、建立内核堆栈、验证参数、内核态与用户态地址空间映射的页面权限更新，以及内核态独立地址空间带来的 TLB 影响。

- **易错/考点**：系统调用不是普通函数调用，开销来自特权级切换、栈切换、参数校验和页表/TLB 更新。

2.3 实验实现（31.4 分钟）

- **主旨**：在 uCore 中动手实现 first-fit 物理页分配算法，以及虚拟地址到页表项的查找与映射建立。
- **关键点**：

1. 练习 1 要求实现 first-fit 连续物理内存分配算法，需补全 default_init、default_init_memmap、default_alloc_pages、default_free_pages 四个函数。
2. 物理内存管理通过 pmm_manager 框架统一抽象，向上暴露 init、init_memmap、alloc_pages、free_pages 四个函数指针，best fit 只是其中一种实现。
3. free_area 数组按 order 分级，每一级挂一条 free_list 双向链表，链表节点就是 page 结构；MAX_ORDER 决定可管理的最大连续页块，用 order 把块挂到对应链表即可完成初始化。
4. 页框总数由 bootloader 传出的内存布局求出最大物理地址，再除以 PGSIZE（pages = maxpa / PGSIZE），据此建立管理所有物理页的 Page 结构数组。
5. 分配时从当前 order 逐级向上找第一个非空 free_list，若块大于所需则反复对半分裂，把剩余半块挂到低一级链表；释放时检查相邻块能否合并，能合并就逐级向上合并。
6. 练习 2 要求实现 get_pte：由页目录基址和线性地址算出 PDE，若页表不存在则分配一页、清零并写入页表项，最后返回对应页表项指针。

- **易错/考点**：释放页后必须 tlb_invalidate 刷新 TLB，且页目录项与页表项的 Present 位检查不可省略；画面以代码/终端为主。

第 3 章　内存管理基础

> 从存储层次讲起，引出两个核心概念：地址空间与地址生成，然后落到最朴素的方案——连续分配。碎片问题在这里第一次出现，后面的分段、分页、虚拟内存，本质上都是在跟碎片和利用率较劲。

3.1 计算机体系结构及内存分层体系（10.4 分钟）

- **主旨**：建立"存储层次"的视角，说明操作系统为什么要抽象和管理内存。
- **关键点**：
  1. 计算机基本硬件结构由 CPU（含运算器、控制器、寄存器）、内存（经内存总线、地址总线连接）、设备（I/O、中断）和磁盘组成，彼此通过总线相连。
  2. 存储层次自上而下为 CPU 寄存器、L1/L2 缓存、主存、磁盘（虚拟内存），越往上越快、越贵、容量越小，越往下越慢、越便宜、容量越大。
  3. 数量级参考：寄存器最快（约 1 个周期、3.6GHz），主存约 1.3GHz，磁盘最慢（约 5ms 寻道时间）；缓存未命中时需要从下一层填充（cache miss/fill）。
  4. 操作系统对内存的四个抽象目标：抽象（逻辑地址空间）、保护（独立地址空间）、共享（访问相同内存）、虚拟化（更多的地址空间）。
  5. 逻辑（虚拟）地址空间经 MMU 映射到物理空间，而物理空间由主存与磁盘上的虚拟内存共同构成。
  6. 操作系统中管理内存的不同方法包括程序重定位、分段、分页、虚拟内存、按需分页虚拟内存。
- **易错/考点**：层次越高越快越贵越小这条规律常考；"虚拟内存"既指磁盘上那块空间，也指一套管理机制，答题时勿混为一谈。

3.2 地址空间与地址生成（11.4 分钟）

- **主旨**：定义物理地址空间与逻辑地址空间，并说明一个程序从源码到装入内存的过程中地址如何生成与重定位。
- **关键点**：
  1. 物理地址空间是硬件支持的地址空间，从 0 到 MAX_phys，由主存和磁盘组成；逻辑地址空间是一个运行的程序所拥有的内存范围，从 0 到 MAX_prog。
  2. 逻辑地址的生成经历四个阶段：编译（c file → s file）、汇编（s file → o file）、链接（o file → exe file）、载入（程序重定位）。
  3. 链接把多个目标文件的代码段、数据段拼接进可执行文件，各段获得统一的偏移；载入时再整体搬到内存中对应的代码段、数据段位置。

4. 物理地址的生成涉及三方：CPU 侧（ALU 需要的指令和数据地址、寄存器中保存的地址、经总线发出逻辑地址）、内存侧（内存控制器从总线取到地址并访问相应单元）、操作系统侧（建立逻辑地址与物理地址之间的映射）。
5. MMU 依靠控制寄存器中的重定位寄存器（base）与界限寄存器（limit）完成地址转换，并顺带做越界检查。
6. 操作系统通过设置逻辑地址空间的 base 与 limit，实现内存共享、内存地址空间保护和内存映射。

- **易错/考点**：地址转换发生在 MMU 而不是 CPU 内部；base/limit 一对寄存器同时承担"映射"与"保护"两种职责。

3.3 连续内存分配：内存碎片与分区的动态分配（12.2 分钟）

- **主旨**：在连续分配的前提下，比较首次匹配、最优匹配、最差匹配三种动态分区策略的规则与优劣。
- **关键点**：

1. 简单的内存管理方法是程序准许运行时分配一个连续区间，操作系统跟踪已分配的满块与空闲块（"孔洞"），由此产生内存碎片问题。
2. 首次匹配（first fit）按地址排序空闲块列表，分配时取第一个足够大的块，剩余部分仍留在原处；优点是简单，且易于把大空闲块保留在地址空间尾部。
3. 首次匹配的劣势是外部碎片和分配时间的不确定性；最优匹配（best fit）则按尺寸排序空闲块，取能满足要求的最小块，以避免分割大空闲块、最小化外部碎片的尺寸。
4. 最优匹配在大部分分配是小尺寸时非常有效且实现简单，但重分配慢，并易产生大量没用的微小碎片。
5. 最差匹配（worst fit）取最大的空闲块来分割，目的是避免产生太多微小碎片；分配很快，但会破碎大空闲块以致大分区无法被分配，中等尺寸分配时效果最好。
6. 三种策略在回收分区时都必须检查能否与相邻空闲分区合并；以 400 字节的分配为例，首次匹配用第一个够大的块，最差匹配用最大的块。

- **易错/考点**：最优匹配并不"最优"，它留下的微小碎片最多；三种算法的空闲块链表是按地址排序还是按尺寸排序，是常考的区分点。

3.4 连续内存分配：压缩式与交换式碎片整理（10.2 分钟）

- **主旨**：给出对付外部碎片的两种整理手段——在内存内搬移程序合并空闲块，以及把等待中的程序换出到磁盘。
- **关键点**：

1. 压缩式碎片整理（compaction）的画面只有内存布局示意图：程序 P1、P2、P3、P4 与空闲块交错分布，箭头表示把程序在内存中搬移，使分散的空闲块合并成连续大块（该页无文字说明）。

2. 交换式碎片整理（swap）针对"运行程序需要更多的内存"，做法是抢占处于等待状态的程序并回收它们占用的内存。
3. 图中 P3 处于运行态并需要更多内存，P1、P2、P4 处于等待态，被抢占程序占用的空间回收进主存空闲区，换出的内容放在磁盘（虚拟内存）上。
4. 交换的代价来自磁盘 I/O，因此本节最后留下问题：哪些程序应该被交换，即换出对象的选择策略。
5. 本章回顾的脉络是：地址空间与地址生成（地址空间定义、地址生成），连续内存分配（内存碎片问题、分区的动态分配——第一适配/最佳适配/最差适配、压缩式碎片整理、交换式碎片整理）。

- **易错/考点**：压缩要搬移数据、有拷贝开销，交换的对象是等待中的程序而不是正在运行的程序（该页仅示意图，未展开细节）。

第 4 章　非连续内存分配

> 用「分段」和「分页」把程序拆散放，从根上消灭外碎片。代价是地址转换变复杂：段表、页表、多级页表、TLB、反向页表，五种结构解决的都是同一个矛盾——转换要快，页表要小。

4.1 非连续内存分配：分段（12.7 分钟）

- **主旨**：针对连续内存分配产生碎片、利用率低的问题，把程序按逻辑划分为段并分散装入不连续的物理内存，用段表完成地址转换。
- **关键点**：
  1. 连续内存分配的三个缺点：分配给一个程序的物理内存必须是连续的、内存利用率较低、存在外碎片与内碎片。
  2. 非连续分配把逻辑地址空间分散映射到多个不连续的物理地址空间，代价是必须额外解决虚拟地址到物理地址的转换，可用软件方案或硬件方案实现。
  3. 段访问机制的新概念：一个段就是一个内存"块"，对应一个逻辑地址空间；访问内存地址需要二维二元组 (s, addr)，s 为段号、addr 为段内偏移。
  4. 段表每项记录该段的段长（Seg Length）和基址（Base），段表本身由操作系统设置和维护。
  5. 硬件实现方案：CPU 给出逻辑地址，MMU 用段号索引段表取出段长与基址，先比较偏移是否越界，再与基址相加得到物理地址；越界等非法访问触发内存异常交由操作系统处理。
  6. 段号与偏移的存放有两种实现：段寄存器 + 地址寄存器方案，以及单地址实现方案。
- **易错/考点**：段式地址是 (段号, 段内偏移) 的二维地址，必须先用偏移与段长比较判越界，再做 base + offset，而不是无条件相加。

4.2 非连续内存分配：分页（12.1 分钟）

- **主旨**：用固定大小的页和帧取代变长的段，把地址转换简化为"页号换成帧号、偏移原样保留"，从根本上消除外碎片。
- **关键点**：
  1. 物理内存被划分为大小固定且相等的帧（Frame），帧大小取 2 的幂（如 512、4096、8192 字节），以便用移位和掩码切分地址。
  2. 物理地址是二元组 (f, o)：f 为帧号（F 位，共 2^F 个帧），o 为帧内偏移（S 位，每帧 2^S 字节），物理地址 = 2^S × f + o。
  3. 逻辑地址空间按同样大小划分为页（Page），页内偏移位数等于帧内偏移位数，页号位数小于等于帧号位数；虚拟地址 = 2^S × p + o。

4. 地址计算实例：16 位地址空间、页大小 512 字节时 S=9、F=7，(f, o)=(3, 6) 代入得物理地址 2^9×3+6 = 1536+6 = 1542。
5. 页寻址机制：页表保存逻辑地址到物理地址的映射，页表基址寄存器指出页表位置，转换时把页号 p 替换为帧号 f，偏移 o 直接拼接。
6. 页是连续的虚拟内存、帧是非连续的物理内存，并且不是所有的页都有对应的帧。

- **易错/考点**：偏移位数由页大小决定（页大小 = 2^S），页号位数 = 地址宽度 – 偏移位数；页是逻辑概念、帧是物理概念，不可混用。

4.3 非连续内存分配：页表－概述、TLB（14.8 分钟）

- **主旨**：讲清页表的组织结构与地址转换流程，指出"两次访存"的性能瓶颈，并用 TLB 缓存转换表项来缓解。
- **关键点**：

1. 每个运行的程序都有一个页表，页表属于程序运行状态、会动态变化，由 PTBR（页表基址寄存器）指出其起始位置。
2. 页表项内容包括帧号 f 和标志位 Flags：dirty bit（脏位）、resident bit（驻留位）、clock/reference bit（访问位）等。
3. 地址转换实例：具有 16 位地址、每页 1024 字节的系统，地址拆为 6 位页号加 10 位偏移，逻辑地址 (4, 0) 经页表换帧号后得到物理地址 (4, 1023)、(0, 0) 等。
4. 分页机制的性能问题：访问一个内存单元需要 2 次内存访问，一次用于获取页表项、一次用于访问数据。
5. 页表可能非常大：64 位机器若每页 1024 字节，页表项数随页数线性增长，单级页表将大到无法直接实现。
6. TLB（快表，Translation Look-aside Buffer）用关联存储器（associative memory）缓存近期访问的页帧转换表项，命中则快速取得物理页号，未命中则查内存页表并把对应表项更新到 TLB 中。

- **易错/考点**：TLB 缓存的是"页号→帧号"的转换表项而不是数据本身；命中时只需 1 次访存，未命中则仍需访问内存中的页表。

4.4 非连续内存分配：页表－二级，多级页表（5.7 分钟）

- **主旨**：把巨大的单级页表拆成按需存在的页表树，用多级间接换取页表空间的节省。
- **关键点**：

1. 二级页表把逻辑地址的页号再拆成 p1、p2 两段，与页内偏移 o 组成三段地址。
2. 一级页表由 PTBR 定位，其表项指向某个二级页表的基址；二级页表项才给出物理帧号 f，偏移 o 直接拼接成物理地址。

3. 只有一级页表需要常驻，二级页表可以在真正用到时才建立，因此不必为整个逻辑地址空间预留连续的页表空间。
4. 多级页表通过把页号分成 k 个部分来实现多级间接，从而建立起页表"树"。
5. 每增加一级页表，地址转换就多一次访存，转换开销随级数增加而增加。

- **易错/考点**：多级页表省的是页表占用的空间、付出的是转换时间，两者是权衡关系；页内偏移 o 的位数在所有级中保持不变。

4.5 非连续内存分配：页表 – 反向页表（11.9 分钟）

- **主旨**：针对大地址空间下前向页表过于庞大的问题，让页表大小与物理地址空间（帧数）对应，并给出关联内存与哈希两种查找方式。
- **关键点**：

1. 大地址空间（64-bits）下前向映射页表变得繁琐，例如需要 5 级页表；反向页表的思路是让页表与物理地址空间的大小对应，而不是与逻辑地址空间对应。
2. 根本原因是逻辑（虚拟）地址空间的增长速度远快于物理地址空间。
3. 页寄存器（Page Registers）方案：每个帧关联一个寄存器，内容包括 Residence bit（此页是否被占用）、Occupier（占用它的进程编号）、Protection bits（保护位）。
4. 页寄存器开销实例：物理内存 4096×4096 = 16 MB、页大小 4 KB、共 4096 个帧，按 8 bytes/register 需 32 KB，额外开销约 32K/16M = 0.2%，而虚拟内存大小可任意。
5. 反向页表以帧号 f 为索引存放 (PID, 页号, 保护位)，因此查找必须同时给出 (PID, p)；可用关联存储器并行比较，也可用哈希表 h(PID, p) 缩小查找范围。
6. 哈希查找步骤：计算哈希函数 f(i) 并以其作为页寄存器表的索引，取出对应页寄存器后检查标签是否包含 i，包含则成功，否则失败。

- **易错/考点**：反向页表项数等于物理帧数而非页数，查找必须携带 PID，且哈希冲突时要比较标签才能确认命中。

第 5 章　虚拟内存

> 内存不够用怎么办？覆盖和交换都不够优雅，于是有了虚拟存储：靠局部性原理，只把正在用的页留在内存。这一章的落点是缺页中断的处理流程和有效访问时间 EAT 的计算。

5.1 虚拟内存的起因（11.2 分钟）

- **主旨**：从程序规模增长与存储器层次"容量—速度—价格"的矛盾出发，说明内存不够用的三种应对路线，引出虚拟存储。
- **关键点**：
  1. 程序规模的增长速度远远大于存储器容量的增长速度：电脑游戏从一代 437K 增长到八代 138M。
  2. 理想中的存储器是"更大、更快、更便宜的非易失性存储器"，现实中并不存在，只能用层次结构折中。
  3. 实际存储器层次：Registers（1 nsec，<1 KB）、Cache（2 nsec，1 MB）、Main memory（10 nsec，64–512 MB）、Magnetic disk（10 msec，5–50 GB）、Magnetic tape（100 sec，20–100 GB），越往下越慢越便宜。
  4. 内存不够用的三种对策：程序太大超过内存容量，用手动覆盖（overlay）技术只把需要的指令和数据放入内存；程序太多超过内存容量，用自动的交换（swapping）技术把暂时不能执行的程序送到外存；想在有限内存中以更小的页粒度装入更大程序，用自动的虚拟存储技术。
  5. 本章路线图：起因 → 覆盖技术 → 交换技术 → 虚存技术（目标、程序局部性原理、基本概念、基本特征、虚拟页式内存管理）。
- **易错/考点**：覆盖由程序员手工完成、以时间换空间；交换以整个进程为单位；虚存由操作系统自动完成且只换入换出部分内容，三者的粒度与自动化程度是常考点。

5.2 覆盖技术（11.0 分钟）

- **主旨**：在物理内存小于程序体积时，靠程序员手工划分模块并让不同时执行的模块共用同一内存区，用时间换空间。
- **关键点**：
  1. 目标是在较小的可用内存中运行较大的程序，常用于多道程序系统，并与分区存储管理配合使用。
  2. 做法是把程序按自身逻辑结构划分为若干功能上相对独立的程序模块，那些不会同时执行的模块共享同一块内存区域，按时间先后运行。
  3. 必要部分（常用功能）的代码和数据常驻内存；可选部分（不常用功能）在其他模块中实现，平时存放在外存，需要时才装入。

4. 不存在调用关系的模块不必同时装入内存，从而可以相互覆盖，即这些模块共用一个分区。
5. 实例：程序 X 共 190K（A 20K、B 50K、C 30K、D 30K、E 20K、F 40K），内存总共只有 110K，划分为 A 的常驻区 20K、覆盖区 0（50K）、覆盖区 1（40K）。
6. 缺点：需要程序员自己划分模块并确定各模块间的覆盖关系，费时费力且增加编程复杂度；覆盖模块从外存装入内存，实际上是以时间延长来换取空间节省。

- **易错/考点**：覆盖的划分与覆盖关系完全依赖程序员，且必须依据模块的调用结构（无调用关系才能互相覆盖），这正是后来虚拟存储要自动化的原因。

5.3 交换技术（9.2 分钟）

- **主旨**：以进程为单位把暂时不能运行的整个地址空间换出到外存、把需要的换入内存，从而腾出内存空间给其他进程。
- **关键点**：

1. 目标是多道程序在内存中时，让正在运行或需要运行的程序获得更多的内存资源；手段是把暂时不能运行的程序送到外存以腾出空闲内存空间。
2. 换出（swap out）把进程整个地址空间保存到外存，换入（swap in）把外存中某个进程的地址空间读入内存，换入换出内容的大小为整个程序的地址空间，交换在内存与后备存储（backing store，磁盘）之间进行。
3. 交换时机的确定：只在内存空间不够或有不够的危险时才换出。
4. 交换区的大小必须足够大，以存放所有用户进程的所有内存映像的拷贝，并且必须能对这些内存映像进行直接存取。
5. 程序换入时的重定位：换出后再换入不一定回到原来的位置，最好采用动态地址映射的方法。
6. 与覆盖的比较：覆盖只能发生在相互之间没有调用关系的模块之间、需程序员给出逻辑覆盖结构；交换以内存中的程序大小为基本单位，主要发生在内存中的程序与管理程序或操作系统之间。

- **易错/考点**：交换的单位是整个进程地址空间而非部分页面，故开销大；正因为换入位置可能变化，必须依靠动态地址映射而不能用静态重定位。

5.4 虚存技术（上）（22.7 分钟）

- **主旨**：在指出覆盖与交换各自缺陷的基础上，给出虚拟存储的目标、程序局部性原理、基本概念与基本特征。
- **关键点**：

1. 覆盖与交换的不足：覆盖需要程序员把整个程序划分为小功能模块并确定覆盖关系，增加程序员负担；交换以进程为交换单位，需把整个地址空间换进换出，增加处理器开销。
2. 虚存目标：像覆盖那样不把程序的所有内容都放在内存中，从而能运行比空闲内存还大的程序，但由操作系统自动完成、无需程序员干涉；像交换那样能在内存与外存之间交换，但只对进程的部分内容

进行交换。

3. 程序局部性原理（principle of locality）：程序在执行过程中的一个较短时期，所执行的指令地址和指令的操作数地址分别局限于一定区域；表现为时间局部性（同一条指令、同一个数据的一次与下次访问集中在较短时间内）和空间局部性（当前指令与邻近几条指令、当前数据与邻近几个数据集中在较小区域）。
4. 局部性原理表明虚拟存储技术从理论上是可以实现的，实现后也应能取得满意效果。
5. 编写方法影响缺页率：页大小 4K、每进程仅分配 1 个物理页、A[1024][1024] 按行存放且每行占一页时，外层 j 内层 i 的按列访问共发生 1024×1024 次缺页中断，而外层 i 内层 j 的按行访问只发生 1024 次。
6. 基本概念与特征：装入部分页面即可启动程序，访问的页不在内存（缺页或缺段）时由处理器通知操作系统调入，暂时不用的页调出到外存；特征是大的用户空间（32 位虚址理论可访问 4GB，而物理内存可能仅 256M）、部分交换、不连续性（物理内存分配不连续、虚拟地址空间使用不连续）。

- **易错/考点**：局部性原理是虚存成立的前提；按行访问与按列访问的缺页次数相差 1024 倍，是"程序编写方式影响缺页率"的经典计算题。

5.5 虚存技术（下）（21.8 分钟）

- **主旨**：落到虚拟页式管理的实现细节：页表表项各标志位的含义、缺页中断处理流程、后备存储的映射来源，以及有效访问时间 EAT 的定量估算。
- **关键点**：

1. 大部分虚拟存储系统采用虚拟页式存储管理，即在页式存储管理基础上增加请求调页和页面置换功能：只装入部分页面即可启动程序，访问的页不在内存时发生缺页中断请求操作系统调入，同时把内存中暂时不用的页面换出到外存。
2. 页表表项由逻辑页号、标志位和页框号组成；驻留位为 1 表示该页在内存、页表项有效，为 0 表示该页还在外存，访问它会导致缺页异常。
3. 修改位（dirty bit）表示该页在内存中是否被修改过，系统回收物理页面时据此决定是否把内容写回外存；访问位记录该页是否被访问过（含读或写），用于页面置换算法；保护位规定只读、可读写、可执行等访问类型。
4. 缺页中断处理：查页表发现页不在内存后陷入操作系统，由 OS 在外存定位该页、分配空闲页框、把页读入内存并更新页表项使其驻留，最后重新执行被中断的指令（各步骤文字较小，部分不清晰）。
5. 后备存储（backing store）解决"未被映射的页存在哪里"：未映射的页保存在二级存储器的交换空间或特殊格式文件中；代码段映射到可执行二进制文件，动态加载的共享库程序段映射到动态调用的库文件，其它段可能映射到交换文件（swap file）。
6. 有效存储器访问时间 EAT = 访存时间 × 页命中率 + page fault 处理时间 × 缺页率；代入访存 10 ns、磁盘访问 5 ms、缺页率 p、脏页率 q，得 EAT = 10(1−p) + 5,000,000p(1+q)。

- **易错/考点**：EAT 中 5 ms 必须换算成 5,000,000 ns；脏页需额外写回，所以公式里多出 q 项，即使缺页率极小，ms 级的磁盘时间也会把平均访问时间拉高几个数量级。

第 6 章　页面置换算法

> 内存满了要换出一页，换谁？OPT 给出理论下界，FIFO 最简单但有 Belady 异常，LRU 用历史近似未来，Clock 用访问位把 LRU 的开销降下来。最后落到工作集与抖动——置换算法解决不了页框不足的根本问题。

6.1 最优页面置换算法（10.3 分钟）

- **主旨**：界定页面置换的功能与目标，并给出以"未来最久不用"为准则的 OPT 算法，作为衡量其他置换算法缺页次数的理论上界。
- **关键点**：
  1. 页面置换的功能：缺页中断发生且内存已满时，须在内存中选择某个物理页面换出，为新页面腾出位置。
  2. 目标是最小化页面换进换出的次数（即缺页中断次数），而未来访问不可知，只能在局部性原理指导下用过去的统计数据预测。
  3. page locking（页面锁定）用页表中的 lock bit 标记必须常驻内存的关键内核代码或 time-critical 应用。
  4. 用页面轨迹（如 3,1,4,2,5,2,1,2,3,4）模拟置换过程并统计缺页数，缺页越少说明性能越好。
  5. OPT 的准则：缺页时对每个驻留页计算"距下一次被访问还要等多久"，选择等待时间最长的页换出。
  6. OPT 在实际系统中无法实现（操作系统无从知道每页何时再被访问），只能作为性能评价基准：在模拟器上记录一遍访问串，第二遍运行时即可套用最优算法。
- **易错/考点**：OPT 不可实现、只作评价基准；判定依据是表底的 time page needed next 行（示例中 a=6、b=5、c=9、d=10），取最大者淘汰。

6.2 先进先出算法（5.4 分钟）

- **主旨**：给出最简单的 FIFO 置换——按页面进入内存的先后次序淘汰最早进入的页面。
- **关键点**：
  1. 系统维护一个链表，记录所有位于内存中的逻辑页；链首是驻留时间最长的页，链尾是驻留时间最短的页。
  2. 缺页时淘汰链首页面，并把新调入的页面添加到链表末尾。
  3. 实现只需一个指针在页框之间循环移动，Frame List 与 Physical Memory 一一对应，开销很小。
  4. 性能较差：被淘汰的页面有可能马上又被访问，属于"经常要访问的页被换出"。
  5. 在 4 个页框、请求串 c a d b e b a b c d 上逐步演示，每次缺页在 Faults 行标出。

- **易错/考点**：FIFO 存在 Belady 现象且很少单独使用；"最早进入内存"不等于"最久未被使用"，这是它与 LRU 的根本差别。

6.3 最近最久未使用算法（11.9 分钟）

- **主旨**：用"最近最久未使用"这一历史信息近似 OPT，使置换决策只需回溯过去而不必预知未来。
- **关键点**：
  1. 缺页时选择最久未被访问的那个页面并将其淘汰。
  2. 依据是程序局部性原理：最近一段时间内被频繁访问的页，在随后的小段时间内还会被再次访问；反之长期未访问的页未来也可能长期不被访问。
  3. 表格用 time page last used 行记录每页上次被访问的时刻（示例 a=2、b=4、c=1、d=3），取数值最小者作为牺牲页。
  4. 实现方式一：维护页链表，最近使用的页在首、最久未用的页在尾；每次访问把该页从链中摘下并移到表头，缺页时淘汰表尾。
  5. 实现方式二：设置活动页栈，访问时把页号压入栈顶，缺页时从栈底取出页面。
  6. 两种实现都要在每次页面访问时调整次序，开销较大。
- **易错/考点**：LRU 只是 OPT 的近似而非等价；开销大正是它难以直接用于实际系统的原因。

6.4 时钟页面置换算法（15.1 分钟）

- **主旨**：用循环链表加访问位，把 LRU 的排序开销降到与 FIFO 同量级，得到折中的 Clock 算法。
- **关键点**：
  1. 需要利用页表项中的访问位（use bit）：页面装入内存时置 0，随后被访问（读/写）时置 1。
  2. 把所有页框组织成环形链表（钟表表面），指针指向最老的页面（最先进入）。
  3. 缺页时扫描：若当前页访问位为 0 则选为牺牲页换出；若为 1 则把该位清 0、指针下移一格继续找，直到找到可淘汰的页。
  4. 伪代码 Clock_Replacement：while (victim page not found) do if used bit == 0 then victim = current page else 清该页 used bit；advance current pointer。
  5. 效果介于 FIFO 与 LRU 之间：给"最近访问过"的页面一次豁免，避免刚用过的页被立即换出。
  6. 在请求串 c a d b e b a b c d 上演示，page table entries 逐列记录各驻留页的 use bit（如 1a、0a、1c、0d），缺页次数少于 FIFO。
- **易错/考点**：Clock 只有一个访问位，无法区分访问的先后与频度，因此只是 LRU 的近似；扫描时"清位"与"指针移动"是同一轮动作，可能需绕行多圈。

6.5 二次机会法（11.7 分钟）

- **主旨**：在 Clock 基础上引入修改位，避免把"脏页"频繁换出而付出写回磁盘的巨大代价。
- **关键点**：
  1. 动机：替换脏页（被修改过的页）必须先写回磁盘，代价巨大，因此希望脏页在一次时钟扫描中尽量被保留下来。
  2. 每个页框用（use bit, dirty bit）两位描述，按 (0,0)→(0,1)→(1,0)→(1,1) 的次序由优先替换到优先保留。
  3. 扫描经过时把 use 位清 0、保留 dirty 位；只有遇到 (0,0) 的页才真正换出。
  4. Enhanced Clock 表中列出扫描前后的状态变化，并标注 (0,0) 一行对应 replace page。
  5. 环形结构里 Page 1—Page 4 各带 use/dirty 两个小格，指针位置决定本轮从哪里开始扫描。
  6. 在请求串 c c d b c e c e a d 上演示，page table entries 中同时记录 use 与 dirty 两位（如 00a、10a、01c、11c）。
- **易错/考点**：二次机会法的"机会"来自访问位被清零后可再绕一圈，而非固定给两次机会；未修改的页（dirty=0）总是比脏页更早被淘汰。

6.6 最不常用法（5.5 分钟）

- **主旨**：以页面的访问次数（频度）而非时间远近作为淘汰依据，得到 LFU 算法。
- **关键点**：
  1. 缺页时选择访问次数最少的那个页面并将其淘汰。
  2. 实现方法：为每个页面设置一个访问计数器，页面每被访问一次计数加 1，缺页时取计数值最小的页。
  3. 主要缺陷：某页在进程开始运行时被密集访问、此后不再使用，却因计数很大而长期占据内存无法被替换。
  4. 计数器需要额外硬件或软件维护，实现代价较高。
  5. 与 LRU 的区别：LRU 考察"多久未被访问"，时间越短越好；LFU 考察"被访问了多少次"，次数越多越好。
- **易错/考点**：LFU 只看累计次数、计数不衰减，可能淘汰刚调入但尚未再次访问的新页；别与 LRU 的"最近"混为一谈。

6.7 Belady现象、LRU、FIFO、Clock的比较（14.4 分钟）

- **主旨**：用同一访问串横向对比三种局部置换算法的缺页数，解释 Belady 异常及其产生原因。
- **关键点**：

1. Belady 现象：采用 FIFO 时，分配给进程的物理页面数增加，缺页率反而升高的异常现象。
2. 访问串 1,2,3,4,1,2,5,1,2,3,4,5：FIFO 在 Frame Size 3 时 Page Fault 为 9，Frame Size 4 时反而为 10。
3. 同一访问串下 LRU：Frame Size 3 时缺页 10 次，Frame Size 4 时 8 次，页框增加不会使缺页增加。
4. LRU 与 FIFO 本质上都是"先进先出"的思路，差别在排序依据：LRU 按最近访问时间动态调整次序，FIFO 按进入内存的时间排序且次序固定不变。
5. 若内存中所有页面都从未被访问过，各页的"最近访问时间"就等于进入内存的时间，LRU 退化为 FIFO。
6. Clock 是二者的折中：实现与 FIFO 一样只需一个指针、开销小，但对被访问过的页能留下标记，性能接近 LRU。

- **易错/考点**：Belady 只出现在 FIFO 这类不考虑访问历史的算法上，LRU/OPT 属于栈式算法不会出现；"LRU 缺页数随页框增加必不增加"是可考结论。

6.8 局部页替换算法的问题、工作集模型（12.5 分钟）

- **主旨**：指出局部置换算法全部依赖局部性原理这一前提，并用工作集模型对局部性做定量刻画。
- **关键点**：

1. 前面各种页面置换算法都基于程序局部性原理；若局部性不成立（如访问序列 1,2,3,...,9 单调递增），在物理页面有限时任何算法每次访问都必然缺页，算法之间也就没有分别。
2. 工作集定义为二元函数 W(t, Δ)：进程在当前时刻 t 之前的 Δ 个时间窗口内所访问过的页面集合。
3. Δ 称为工作集窗口（working set window），是一个定长的页面访问时间窗口；|W(t, Δ)| 为工作集大小，即页面数目。
4. W(t, Δ) 随 t 滑动而不断变化；示例访问串 261577775162341234443434441327，Δ=10 时 W(t1,Δ)={1,2,5,6,7}、W(t2,Δ)={3,4}。
5. 工作集大小的变化规律：进程启动后随访问页面增多而上升，经过渡阶段后在稳定阶段围绕某个数值上下波动。
6. 常驻集（resident set）指当前时刻进程实际驻留在内存中的页面集合；工作集是进程运行中固有的性质，常驻集则取决于系统分配的物理页面数目和所用置换算法。

- **易错/考点**：工作集是"窗口内访问过的页"，常驻集是"此刻在内存的页"，二者不等；只有常驻集 ⊇ 工作集时进程才能顺利运行。

6.9 两个全局置换算法（19.0 分钟）

- **主旨**：把页框分配由固定改为可变，给出工作集页置换与缺页率置换（PFF）两种全局置换算法。
- **关键点**：

1. 工作集页置换算法：追踪之前 τ 个引用构成工作集，τ 称为窗口大小；页面一旦滑出窗口，即使未被"淘汰"也应从内存中换出。
2. 示例取 τ=4、请求串 c c d b c e c e a d，逐列标出窗口覆盖的页，据此决定驻留集内容。
3. 缺页率（PFF, page fault frequency）用"缺页次数 / 内存访问次数"的比率，或用缺页平均时间间隔的倒数来度量。
4. PFF 的调节策略：缺页率高时增加工作集（多分配页框），缺页率低时减少工作集。
5. PFF 的判定：若 t_current – t_last 大于阈值 T，认为间隔"大"，把 [t_last, t_current] 内未被引用的页从工作集中移除；若小于 T，只把本次缺失页加入工作集。
6. 影响缺页率的因素包括页面置换算法、分配给进程的物理页面数目、页面本身的大小、程序的编写方法。

- **易错/考点**：全局置换允许从其他进程处夺取页框，故常驻集大小可变；PFF 示例中 window size=2，表底 t_cur–t_last 为 1、3、2、3、1，据此逐列增减驻留页。

6.10 抖动问题（8.2 分钟）

- **主旨**：说明页框不足引发抖动（thrashing）的机理，并给出以缺页率为依据的多道程序度控制方法。
- **关键点**：

1. 抖动：分配给进程的物理页面太少，无法包含整个工作集（常驻集 ⊂ 工作集），页面在内存与外存之间频繁换进换出，进程运行速度急剧变慢。
2. 产生原因：随着驻留内存的进程数目增加，分给每个进程的物理页面数不断减少，缺页率不断上升。
3. 因此操作系统必须选择适当的进程数目与每进程所需帧数，在并发水平和缺页率之间取得平衡。
4. 更合理的控制量是多道程序度 MPL（multiprogramming level），判据可取 page fault rate/(1 – page fault rate) 与 MTBF/PFST 比较，也可取 1/内存大小。
5. CPU 利用率随 MPL 先上升、越过峰值后下降；图中给出 N_max 与 N_IO_BALANCE 两个关键点，后者由 MTBF/PFST 曲线与 CPU 曲线的交叉确定。

- **易错/考点**：抖动不是置换算法选错，而是页框分配不足或并发度过高；此时继续提高 MPL 只会让 CPU 利用率进一步下降。

第 7 章　进程与线程

> 进程是「程序 + 执行 + 资源」的三合一抽象。本章从定义、组成、PCB、状态模型讲到挂起，再引出线程（为什么要有、怎么实现），最后落到 fork/exec/wait/exit 四个控制原语。这是全课出现频率最高的一章。

7.1 进程的定义（5.0 分钟）

- **主旨**：从操作系统需要管理并发运行的程序出发，给出进程的定义并说明其地址空间是如何形成的。
- **关键点**：
  1. 本章进程管理分六部分：进程（PROCESS）描述、进程状态（State）、线程（THREAD）、进程间通信（INTER-PROCESS COMMUNICATION）、进程互斥与同步、死锁（DEADLOCK）。
  2. 进程的定义是「一个具有一定独立功能的程序在一个数据集合上的一次动态执行过程」。
  3. 任务管理器截图显示系统中同时驻留大量进程与 CPU/内存曲线，用「Why processes?」引出为并发执行建立抽象的必要性。
  4. 源程序经 Compile（编译）生成 Executable File（可执行文件），文件内部由 Header、Code、Initialized data 三部分组成。
  5. 可执行文件经 Load（装入）形成进程的地址空间（Process's address space），包含 mapped segments（映射段）、DLL's（动态链接库）、Stack（栈）、Heap（堆）、Initialized data、Code。
- **易错/考点**：定义中的「一次动态执行」是关键字，同一可执行文件被多次装入即产生多个互不相同的进程。

7.2 进程的组成（9.8 分钟）

- **主旨**：说明一个进程由哪些成分构成，并从联系与区别两个角度厘清进程与程序的关系。
- **关键点**：
  1. 一个进程应包括：程序的代码、程序处理的数据、程序计数器中的值（指示下一条将运行的指令）、一组通用寄存器的当前值（栈、堆）、一组系统资源（如打开的文件）。
  2. 概括而言，进程包含了正在运行的一个程序的所有状态信息。
  3. 二者联系：程序是产生进程的基础；程序的每次运行构成不同的进程；进程是程序功能的体现；通过多次执行一个程序可对应多个进程，通过调用关系一个进程可包括多个程序。
  4. 二者区别：进程是动态的、程序是静态的；程序是有序代码的集合，进程是程序的执行且分核心态/用户态；进程是暂时的、程序是永久的，程序可长久保存。
  5. 组成不同：进程包括程序、数据和进程控制块（即进程状态信息），程序只是代码集合。

6. 类比：食谱 = 程序，科学家 = CPU，原料 = 数据，做蛋糕 = 进程；CPU 从做蛋糕切换到医疗救护，即进程切换。

- **易错/考点**：进程不等于「程序的一次执行」，还必须有数据与 PCB；「一个程序对应多个进程」与「一个进程包含多个程序」两个方向都成立，不要只记单向。

7.3 进程的特点（6.9 分钟）

- **主旨**：归纳进程区别于程序的四个基本特征，并指出描述进程所用的数据结构。
- **关键点**：

1. 动态性：进程可动态地创建、结束。
2. 并发性：进程可以被独立调度并占用处理机运行，可并发执行。
3. 独立性：不同进程的工作不相互影响。
4. 制约性：因访问共享资源/数据或进程间协同而互相制约。
5. 并发执行要求每个进程有独立的程序计数器，图中由 One program counter 经 Process switch 变为 Four program counters（进程 A、B、C、D 各自推进）。
6. 程序 = 算法 + 数据结构；描述进程的数据结构是进程控制块（Process Control Block, PCB），操作系统为每个进程维护一个 PCB 以保存其各种状态信息。

- **易错/考点**：独立性与制约性并不矛盾——独立性指地址空间与工作互不干扰，制约性指共享资源与协同时的相互牵制，两者同时成立。

7.4 进程控制结构（7.3 分钟）

- **主旨**：讲清 PCB 是什么、包含哪三类信息、以何种方式组织，从而支撑操作系统对进程的管理与调度。
- **关键点**：

1. 进程控制块是操作系统管理控制进程运行所用的信息集合，用来描述进程的基本情况以及运行变化的过程，PCB 是进程存在的唯一标志。
2. 使用 PCB 的三个环节：创建进程时为其生成一个 PCB；终止进程时回收其 PCB；进程的组织管理通过对 PCB 的组织管理来实现。
3. 第一类为进程标识信息：本进程的标识、本进程产生者的标识（父进程标识）、用户标识。
4. 第二类为处理机状态信息保存区，保存运行现场：用户可见寄存器（用户程序可用的数据、地址等寄存器）、控制和状态寄存器（程序计数器 PC、程序状态字 PSW）、栈指针（过程调用/系统调用/中断处理和返回时使用）。
5. 第三类为进程控制信息：调度和状态信息、进程间通信信息（标识、信号、信件等，存于接收方 PCB）、存储管理信息（指向本进程映像存储空间）、进程所用资源（如打开的文件）、有关数据结构连接信息（链入进程队列或关联其他进程的 PCB）。

6. PCB 的两种组织方式：链表——同一状态的 PCB 成一链表，形成就绪链表、阻塞链表；索引表——同一状态的进程归入一个 index 表（由 index 指向 PCB），形成就绪索引表、阻塞索引表。

- **易错/考点**：PCB 不只是「进程号」，而是标识、现场、控制三类信息的集合；进程间通信的信件等存放在接收方 PCB 中，这一点常被忽略。

7.5 进程的生命期原理（7.2 分钟）

- **主旨**：沿创建、运行、等待、唤醒、结束五个阶段，说明进程从产生到消亡的生命期管理。
- **关键点**：

1. 生命期管理包含五个环节：进程创建、进程运行、进程等待、进程唤醒、进程结束。
2. 引起创建的三类事件：系统初始化时、用户请求创建一个新进程、正在运行的进程执行了创建进程的系统调用；创建后由 Start 转入 Ready。
3. 进程运行指内核选择一个就绪的进程让它占用处理机并执行，需要回答「为何选择」与「如何选择」两个问题。
4. 进程等待（阻塞）的三种情形：请求并等待系统服务无法马上完成、启动某种操作无法马上完成、需要的数据没有到达；进程只能自己阻塞自己，因为只有进程自身才知道何时需要等待某事件发生。
5. 唤醒进程的三个原因：被阻塞进程需要的资源可被满足、被阻塞进程等待的事件到达、将该进程的 PCB 插入到就绪队列；进程只能被别的进程或操作系统唤醒。
6. 进程结束的四种情形：正常退出（自愿的）、错误退出（自愿的）、致命错误（强制性的）、被其他进程所杀（强制性的）。

- **易错/考点**：阻塞只能由进程自身发起、唤醒只能由外部（其他进程或 OS）完成，两个方向相反，是本节最典型的判断点。

7.6 进程状态变化模型（7.1 分钟）

- **主旨**：建立进程的三基本状态模型，并逐一说明各状态之间转换的触发条件。
- **关键点**：

1. 运行状态（Running）：当一个进程正在处理机上运行时。
2. 就绪状态（Ready）：进程已获得除处理机之外的一切所需资源，一旦得到处理机即可运行。
3. 等待状态（又称阻塞状态 Blocked）：进程正在等待某一事件而暂停运行，如等待某资源、等待输入/输出完成。
4. 进程在生命结束前处于且仅处于三种基本状态之一，不同系统设置的进程状态数目不同。
5. 经典三态图用四个编号表示转换：1 Process blocks for input、2 Scheduler picks another process、3 Scheduler picks this process、4 Input becomes available。

6. 扩展为含 New/Exit 的状态变化图：New 创建后进 Ready；Running 因超时或被剥夺回 Ready；Running 等待事件进 Blocked；Blocked 因事件发生回 Ready；Running 退出进 Exit。

- **易错/考点**：就绪与阻塞的判别标准是「只差处理机」还是「还差事件/资源」；阻塞进程不可被调度，而就绪进程被选中才真正占用处理机。

7.7 进程挂起（9.8 分钟）

- **主旨**：说明为什么要引入挂起状态、挂起与解挂引起的状态转换，以及用状态队列组织 PCB 的方法。
- **关键点**：

1. 挂起的目的是合理且充分地利用系统资源：进程处于挂起状态时不再占用内存空间，其进程映像存放在磁盘上。
2. 两种挂起态：阻塞挂起状态（Blocked-suspend）——进程在外存并等待某事件出现；就绪挂起状态（Ready-suspend）——进程在外存，但只要进入内存即可运行。
3. 挂起（Suspend）把进程从内存转到外存，三种情形：没有就绪进程或就绪进程要求更多内存时，挂起阻塞进程以提交新进程或运行就绪进程；高优先级阻塞进程与低优先级就绪进程并存时，挂起低优先级就绪进程；抢先式分时系统中高优先级阻塞挂起进程因事件出现而进入就绪挂起时，可能把运行进程转为就绪挂起。
4. 外存内的转换：阻塞挂起进程的相关事件出现时，系统将其转为就绪挂起进程。
5. 解挂/激活（Activate）把进程从外存转到内存：没有就绪进程或挂起就绪进程优先级高于就绪进程时，就绪挂起转就绪；进程释放足够内存时，把高优先级阻塞挂起进程转为阻塞进程。
6. 状态队列：操作系统维护一组队列表示所有进程的当前状态（就绪队列、各种类型的阻塞队列），进程状态变化时其 PCB 从原队列脱离并加入另一队列；就绪队列经调度进入运行，运行超时回就绪，等待事件进入阻塞队列，最终终止。

- **易错/考点**：挂起不等于阻塞——阻塞是等事件但仍驻留内存，挂起是被换出到外存；就绪挂起被激活后进入就绪队列而不是直接运行。

7.8 为什么使用线程（5.4 分钟）

- **主旨**：用 MP3 播放器案例对比单进程与多进程两种实现，说明二者都无法同时满足"并发执行"与"共享地址空间"，从而引出线程这一新实体。
- **关键点**：

1. 单进程实现把 Read()（读音频数据）、Decompress()（解压）、Play()（播放）顺序放在 main 的 while(TRUE) 循环里，I/O 与 CPU 串行执行。
2. 单进程的后果是播放出来的声音断断续续，且各函数之间不能并发执行，影响资源利用率。
3. 多进程实现拆成程序1、程序2、程序3 三个进程分别只做 Read、Decompress、Play，但进程之间必须通信才能共享数据。

4. 多进程的开销在于：创建进程时要分配资源、建立 PCB（进程控制块）；撤销进程时要回收资源、撤销 PCB；进程切换时要保存当前进程的状态信息。
5. 解决方案要求新实体满足两条特性：实体之间可以并发执行；实体之间共享相同的地址空间。

- **易错/考点**：单进程"声音断续"与多进程"通信+切换开销"是两个不同层面的缺陷，线程正是为同时满足并发执行与地址空间共享而提出的。

7.9 什么是线程（12.2 分钟）

- **主旨**：从资源组合与执行流程两个角度重新理解进程，给出线程的定义以及"线程 = 进程 – 共享资源"的关系。
- **关键点**：

1. Thread（线程）是进程当中的一条执行流程。
2. 从资源组合角度：进程把一组相关资源组合到一个进程平台（PCB）上，包括地址空间中的 Code（代码段）、Initialized data（已初始化数据）、Zero filled data（零初始化数据）、Heap（堆）。
3. 从运行角度：进程是这个平台之上的一条执行流程（线程），同一进程的多个线程各有独立的 Stack（栈）和 TCB（线程控制块），但共用 Code、Data、Heap。
4. 线程 = 进程 – 共享资源：线程独享栈与寄存器等执行上下文，共享地址空间和文件等资源。
5. 线程优点：一个进程中可同时存在多个线程；各线程之间可并发执行；各线程之间可共享地址空间和文件等资源。
6. 线程缺点：一个线程崩溃，会导致其所属进程的所有线程崩溃。

- **易错/考点**：线程私有的是栈和 TCB/寄存器上下文，共享的是代码段、数据段、堆与打开的文件；"一个线程崩溃拖垮整个进程"是线程模型的固有代价。MS-DOS 单线程、OS/2 与 Windows NT 对线程的支持差异（画面不清晰）。

7.10 线程的实现（9.1 分钟）

- **主旨**：给出线程的三种实现方式（用户线程、内核线程、轻量级进程），并剖析用户线程与内核线程的机制和各自代价。
- **关键点**：

1. 三种实现方式：用户线程在用户空间实现（POSIX Pthreads、Mach C-threads、Solaris threads）；内核线程在内核中实现（Windows、Solaris、Linux）；轻量级进程在内核中实现并支持用户线程（Solaris 的 LightWeight Process）。
2. 用户线程由一组用户级线程库函数完成创建、终止、同步和调度，不依赖操作系统内核，也不需内核了解用户线程的存在，因此可用于不支持线程技术的多进程操作系统。
3. 每个进程维护自己私有的 TCB（Thread Control Block，线程控制块）列表，记录各线程的 PC、栈指针、寄存器等状态；图中 TCB 位于用户空间，内核侧只有 Run-time system、Thread table、

Process table 与 PCB。

4. 用户线程的切换由线程库函数完成，无需用户态/核心态切换，速度特别快，并允许每个进程拥有自定义的线程调度算法。
5. 用户线程缺点：一个线程发起阻塞性系统调用会使整个进程等待；线程不主动交出 CPU，则同进程其他线程无法运行；时间片按进程分配，多线程时每个线程分到的时间片更少、执行更慢。
6. 内核线程由内核维护 PCB 与 TCB，创建、终止、切换都通过系统调用/内核函数完成，开销较大，但某线程阻塞不影响其他内核线程，且时间片按线程分配使多线程进程获得更多 CPU 时间。

- **易错/考点**：用户线程"切换快"却会因一个线程阻塞而拖住整个进程、且无法真正并行；内核线程"开销大"但可独立阻塞、可并行调度。轻量级进程是内核支持的用户线程，一个 LWP 由一个单独的内核线程支撑。

7.11 上下文切换（4.9 分钟）

- **主旨**：说明进程切换时必须保存与恢复上下文，以及为什么这一动作既要完整又要尽可能快。
- **关键点**：

1. 上下文切换指停止当前运行进程（从运行状态改变成其他状态），并且调度其他进程转变成运行状态。
2. 必须在切换之前存储当前进程的上下文，并且之后能够恢复，使进程不能显示出它曾经被暂停过。
3. 需要存储的上下文包括寄存器（PC、SP 等）和 CPU 状态等。
4. 切换必须快速，因为上下文切换非常频繁；其代价可能很昂贵，所以必须尽可能避免。
5. 切换流程（P0→P1）：P0 executing → interrupt or system call → save state into PCB0 → reload state from PCB1 → P1 executing，P0 转为 idle、P1 由 idle 转 executing，反向切换过程对称。
6. 片尾引出：操作系统为活跃进程准备 PCB（进程控制块），并把 PCB 放入合适的队列，如按设备划分的 I/O 队列和就绪队列。

- **易错/考点**：保存的是当前进程上下文到它自己的 PCB，恢复的是另一个进程 PCB 中的状态；上下文切换本身不产生有用工作，是纯开销，因此要减少切换次数。

7.12 进程控制——创建进程（4.9 分钟）

- **主旨**：本节本应说明操作系统如何创建新进程（fork 相关机制），但所提供的 8 帧画面全部与上一节《上下文切换》重复，本节正文无法辨认。
- **关键点**：

1. 8 帧时间戳依次为 0:18、0:54、1:31、2:07、2:44、3:20、3:57、4:54，与 v059《上下文切换》的缩略图逐帧一致，判定为抽帧文件重复，本节实际画面缺失。

2. 可辨认的 2:44—3:57 三帧仍是上下文切换流程图：进程经 interrupt or system call 触发 save state into PCB，再由 reload state from PCB 恢复另一进程。
3. 4:54 一帧提到操作系统为活跃进程准备 PCB（进程控制块）并放入合适的队列，含按设备划分的 I/O 队列与就绪队列（画面不清晰）。

- **易错/考点**：本节画面无法辨认，无可提取考点；创建进程的具体机制可参考 7.13 中 fork() 的简单实现及 fork 与 exec 的配合。

7.13 进程控制——加载和执行进程（11.4 分钟）

- **主旨**：说明 exec() 如何在不新建进程的前提下，用新程序覆盖当前进程的代码、栈和堆，以及它为何总是跟在 fork() 之后调用。
- **关键点**：

1. 系统调用 exec() 加载程序取代当前运行的进程：允许一个进程加载一个不同的程序并在 main 开始执行（事实上是 _start）。
2. exec() 允许进程指定参数的数量（argc）和字符串参数数组（argv）。
3. 如果调用成功，它仍是相同的进程，但运行的是一个不同的程序；代码、stack（栈）、heap（堆）被重写。
4. 地址空间图示：替换前 shell 进程 pid=127、open files="/bin/sh"、last_cpu=0，Code 为 int shell main(){ int a = 2; }，Stack 中 a=2，Heap 起始 0xPC0933CA；exec 后 pid 仍为 127 而 Code 变为 int calc main(){ int q = 7; }，Heap 起始变为 0x43178050。
5. fork() 的简单实现：为子进程分配内存，复制父进程的内存内容（地址空间），然后开始执行；fork 后父子进程 pid 不同（127 与 128），open files 可分别指向 /bin/sh 与 /bin/calc。
6. UNIX 的情况是在 fork() 之后调用 exec()：fork() 在父进程和子进程中重复运行，子进程可先关闭继承的打开文件和网络连接再开始执行新程序；Windows 则把创建与加载合并为一次调用；现代实现用 Copy on Write（COW，写时复制）避免复制整个地址空间（部分文字画面不清晰）。

- **易错/考点**：exec 不创建新进程（pid 与 PCB 不变，只换程序映像），fork 才创建新进程；fork 负责复制、exec 负责覆盖，二者配合完成"创建并运行另一个程序"。

7.14 进程控制——等待和终止进程（9.9 分钟）

- **主旨**：说明父进程用 wait() 回收子进程的返回值、子进程用 exit() 结束自身，以及由此产生的 zombie（僵尸）状态与进程状态转换全图。
- **关键点**：

1. wait() 系统调用是被父进程用来等待子进程结束的：子进程会向父进程返回一个值，所以父进程必须接收这个值并处理。

2. wait() 使父进程睡眠以等待子进程结束；当子进程调用 exit() 时，操作系统唤醒父进程，并把 exit() 传递的返回值连同子进程的 pid 一起作为 wait() 的结果；若没有存活的子进程，wait() 立刻返回。
3. 当存在已终止但未被回收的子进程时，wait() 立即返回其中一个的值并解除阻塞、释放其资源（画面不清晰）。
4. exit() 以进程的"结果"作为一个参数，关闭所有打开的文件和管道等，释放内存，释放大部分支持进程的操作系统结构。
5. exit() 检查父进程是否存活：父进程存活则结果值交给父进程，此时进程并未真正死亡，而是进入 zombie（僵尸）状态；父进程已不在则清理全部资源，该进程真正死亡；同时清理它所有等待的子进程。
6. 进程状态图：New —fork()→ Running；Running —exit()→ Zombie —parent wait()→ Exit；Running 经"等待事件"转入 Blocked，Blocked 经"事件发生"回到 Ready，Ready 经"调度"回到 Running。

- **易错/考点**：僵尸态是"已终止但退出码与 PCB 尚未被回收"，只有父进程 wait() 才能让它进入 Exit；进程终止是最终的垃圾收集（资源回收）。

第 8 章　CPU 调度

> 就绪队列里挑谁上 CPU。评价指标互相打架（响应快 vs 开销小 vs 公平），于是有了 FCFS、SPN/SRT、HRRN、轮转、多级反馈队列，再延伸到实时调度（RM/EDF）与多处理器下的优先级反转。

8.1 背景（7.5 分钟）

- **主旨**：说清 CPU 调度要解决的是"从就绪队列里挑谁上 CPU"，以及这种挑选发生在进程生命周期的哪些时刻。
- **关键点**：
  1. 上下文切换（context switch）包含三件事：切换 PCB 中的当前任务（从一个进程/线程到另一个）、保存当前进程/线程在 PCB/TCB 中的执行上下文（CPU 状态）、读取下一个进程/线程的上下文。
  2. CPU 调度就是从就绪队列中挑选一个进程/线程作为 CPU 将要运行的下一个进程/线程，调度程序是挑选进程/线程的内核函数，受一套调度策略约束。
  3. 进程/线程生命周期由 Start、Ready、Running、Waiting、Done 五个状态构成，调度点分布在这些状态迁移处。
  4. 内核运行调度程序的条件（满足一条即可）：一个进程从运行状态切换到等待状态，或者一个进程被终结了——这两条属于"不可抢占"。
  5. "可抢占"的条件更宽：调度程序在中断被响应后执行、当前进程从运行切换到就绪（或另一个进程从等待切换到就绪）、当前运行的进程可以被换出。
- **易错/考点**：非抢占只发生在"运行→等待"和"运行→终止"，而"运行→就绪"（时间片到）与中断返回都属于抢占点。

8.2 调度原则（13.0 分钟）

- **主旨**：建立评价调度算法的指标体系，并说明"更快""公平"这类目标本身并不唯一。
- **关键点**：
  1. 执行模型：程序在 CPU 突发（CPU burst）和 I/O 之间交替，每次调度决定的是下一个 CPU 突发交给哪个工作；时间片机制下，进程可能在当前 CPU 突发结束前被剥夺 CPU。
  2. 五个指标：CPU 使用率＝CPU 处于忙状态所占时间的百分比；吞吐量＝单位时间内完成的进程数量；周转时间＝进程从初始化到结束（含所有等待时间）所花的时间；等待时间＝进程在就绪队列中的总时间；响应时间＝从请求提交到产生第一次响应所花的总时间。
  3. "更快"是两个独立因素：传输文件要的是高带宽，玩游戏要的是低延迟，二者不可互相替代。

4. 水管类比：低延迟像一开水龙头水就流出来；高带宽像挖一条粗水管，希望从水龙头里同时流出大量水，并不介意是否存在延迟。
5. 调度目标：减少响应时间、减少平均响应时间的波动（交互系统中可预测性比高低更重要）、增加吞吐量（减少操作系统开销如上下文切换、提高 CPU 与 I/O 设备利用率）、减少等待时间。
6. 公平有两种互不相同的定义：保证每个进程占用相同的 CPU 时间，或保证每个进程都等待相同的时间；公平通常会增大平均响应时间。

- **易错/考点**：吞吐量对应"计算带宽"、响应时间对应"计算延迟"，低延迟调度提升交互表现但必须保证吞吐量不受影响——这是两者的取舍关系。

8.3 调度算法1（19.8 分钟）

- **主旨**：给出 FCFS、SPN/SRT、HRRN 三类非时间片算法的规则，并用平均响应时间的计算说明顺序为何重要。
- **关键点**：

1. 算法清单：FCFS（First Come, First Served，先来先服务）、SPN/SJF/SRT（短进程优先/短作业优先/最短剩余时间优先）、HRRN（Highest Response Ratio Next，最高响应比优先）、Round Robin（轮转）、Multilevel Feedback Queues（多级反馈队列）、Fair Share Scheduling（公平共享调度）。
2. FIFO 队列规定：如果进程在就绪队列队首，则把 CPU 给它；单实例下三个进程计算时间 12、3、3，按 P1、P2、P3 到达时平均响应时间为 (12+15+18)/3=15，改为 P2、P3、P1 顺序则降为 (3+6+18)/3=9。
3. SPN 每次都选预期下次 CPU 突发最短的进程，并按预期突发时间把新任务插入队列；它可以是抢占式也可以是非抢占式，抢占式即 SRT（Shortest Remaining Time，最短剩余时间）。
4. SJF 具有最优平均等待时间：平均响应时间按 (r1+r2+r3+...+rn)/n 计算，短任务排在长任务之前即可把总和压到最小，故改变执行顺序能降低平均响应时间。
5. 短任务优先会让长任务饥饿（starvation），而且需要预知未来的 CPU 突发长度；实际用指数平均预测：$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$，展开为 $\alpha t_n+(1-\alpha)\alpha t_{n-1}+(1-\alpha)(1-\alpha)\alpha t_{n-2}+\ldots$，其中 $0\le\alpha\le1$。
6. HRRN 在 SPN 基础上改进、不可抢占，关注进程已经等待了多长时间，响应比 $R=(w+s)/s$（w 为 waiting time 等待时间，s 为 service time 执行时间），选择 R 值最高的进程以防止无限期推迟。

- **易错/考点**：SRT 是 SPN 的抢占版本，二者不是两种独立算法；HRRN 的分子是"等待+执行"，等待越久响应比越大，这正是它防饥饿的机制。

8.4 调度算法2（17.1 分钟）

- **主旨**：讲轮转与多级反馈队列在"响应快"和"开销小"之间的权衡，并给出评价调度算法的方法。
- **关键点**：

1. RR（Round Robin，轮转）例题中 P1=53、P2=8、P3=68、P4=24，四个进程的平均等待时间为 (72+20+85+88)/4=66.25，其代价是额外的上下文切换花销。
2. 时间量子太大导致等待时间过长，极端情况下退化成 FCFS；时间量子太小则响应迅速，但吞吐量会因大量上下文切换受影响，目标是选一个合适的时间量子，经验规则是把上下文切换开销维持在 1% 以内。
3. 对比表中同一组进程在 FCFS、RR（q=1 至 q=50）、Shortest Process、Word FCFS 下的平均等待时间依次变化，说明量子取值直接决定性能排序。
4. MLFQ（Multilevel Feedback Queues，多级反馈队列）把就绪队列划分为多个独立的队列，每个队列拥有自己的调度策略（前台交互用 RR、后台批处理用 FCFS），且调度必须在队列之间进行。
5. 队列之间可用固定优先级（先处理前台、后处理后台，可能导致饥饿），也可用时间切片（每个队列分到能调度其进程的 CPU 时间，如 80% 给前台 RR、20% 给后台 FCFS）；进程可在不同队列间移动，时间量子小则优先级升高，时间片用完仍未完成则降到下一优先级，从而使 I/O 密集型任务停留在高优先级、CPU 密集型任务降到低优先级。
6. FFS（Fair Share Scheduling，公平共享调度）控制用户对系统资源的访问：保证不重要的组无法垄断资源、未达到资源使用目标的组获得按比例分配、资源使用率越高的组获得更多优先权；评价算法可用确定性建模、队列模型、实现/模拟三种方法，其中模拟最灵活也最昂贵。

- **易错/考点**：MLFQ 的"时间切片"指队列之间按比例分配 CPU，不是进程内的时间片；量子过小并非"更好"，切换开销会吃掉吞吐量。

8.5 实时调度（8.7 分钟）

- **主旨**：界定实时系统的正确性标准，并给出 RM 与 EDF 两种优先级分配方式。
- **关键点**：

1. 实时操作系统是正确性依赖于时间和功能两方面的一种操作系统，性能指标是时间约束的及时性（deadlines），速度和平均性能相对不重要，主要特性是时间约束的可预测性。
2. 强实时系统需要在保证的时间内完成重要的任务、必须完成；弱实时系统只要求重要进程的优先级更高、尽量完成，并非必须。
3. 任务是一系列相似的工作单元（一次计算、一次文件读取、一次信息传递等），属性包括取得进展所需要的资源与定时参数：Released（释放时刻）、Execution time（执行时间）、Relative deadline（相对截止期）、Absolute deadline（绝对截止期）。
4. 周期任务有规律地重复，周期 p 为 inter-release time（相邻两次释放的间隔），执行时间 e 为最大执行时间，使用率 U=e/p。
5. 可调度性分析用来表示一个实时系统是否能够满足 deadline 要求，并据此决定实时任务执行的顺序，分为静态优先级调度与动态优先级调度。
6. RM（Rate Monotonic，速率单调）是最佳静态优先级调度，通过周期安排优先级——周期越短优先级越高、先执行周期最短的任务；EDF（Earliest Deadline First，最早期限优先）是最佳动态优先级

调度，deadline 越早优先级越高、先执行 deadline 最早的任务。

- **易错/考点**：RM 的优先级只由周期决定（与执行时间无关），且"最佳"仅限于静态优先级范畴；EDF 的"最佳"属于动态优先级范畴，两者不能互相替代。

8.6 多处理器调度与优先级反转（8.6 分钟）

- **主旨**：说明 SMP 下调度为何比单处理器复杂，以及优先级反转的成因与两种修复手段。
- **关键点**：
  1. 多处理器的 CPU 调度更加复杂：由多个相同的单处理器组成一个多处理器，优点是可以负载共享。
  2. SMP（Symmetric Multiprocessor，对称多处理器）中每个处理器运行自己的调度程序，因此需要在调度程序之间进行同步。
  3. 优先级反转可以发生在任何基于优先级的可抢占的调度机制中，当系统内的环境强制使高优先级任务等待低优先级任务时发生，火星探路者（Mars Pathfinder）就是典型实例。
  4. 无限制优先级反转的持续时间取决于其他不相关任务的行为：图中 T1 高优先级任务被 T3 持有的锁阻塞，而 T2 的抢占不断延长 T3 的剩余执行时间，导致阻塞时间不可预测。
  5. 优先级继承：低优先级任务继承高优先级任务的优先级，其优先级高低取决于它们所共享的资源。
  6. 优先级天花板：把"资源"的优先级定义为所有可以锁定该资源的任务中优先级最高的那个任务的优先级；除系统内所有被锁定的资源之外，任务尝试执行临界区时会被阻塞，持有最高优先级上限信号量锁的任务则继承被该锁所阻塞任务的优先级。
- **易错/考点**：优先级反转的根因是共享资源上的互斥等待被中等优先级任务"插队"放大，不是调度算法本身写错；继承是运行时临时提升，天花板是给资源预设固定上限。

第 9 章　同步与互斥

> 并发为什么危险？因为语句不是原子的。本章先建立 race condition、临界区、互斥、死锁、饥饿这套术语，再依次试三种方案：关中断、纯软件算法（Peterson/Bakery）、硬件原子指令。结论是——都需要一个更好的抽象。

9.1 背景知识（13.0 分钟）

- **主旨**：用 fork() 分配 PID 的例子说明并发执行的不确定性，以及为什么并行程序必须先设计再编码。
- **关键点**：
  1. 独立的线程不与其他线程共享资源或状态，因此具有确定性（输入状态决定结果）、可重现（能重现起始条件即可复现）、调度顺序不重要。
  2. 合作的线程在多个线程间共享状态，因此不确定、不可重现，其 bug 可能是间歇性发生的。
  3. 进程/线程、计算机/设备需要合作的三个优点：共享资源（一台电脑多个用户、一个银行存款多台 ATM、嵌入式系统人机控制）、加速（I/O 操作和计算可以重叠、多处理器可将程序分成多个部分并行执行）、模块化（把大程序分解成小程序，如 gcc 调用 cpp、cc1、cc2、as、ld）。
  4. 程序可调用 fork() 创建一个新的进程：操作系统需要分配一个新的并且唯一的进程 ID，因此在内核中该系统调用运行、共享的全局变量 next_pid 会被修改（原子操作）。
  5. 汇编层面该调用展开为 LOAD next_pid, Reg1 / STORE Reg1, new_pid / INC Reg1 / STORE Reg1, next_pid；若 next_pid 初值为 100，则一个进程得到的 new_pid 应为 100，另一个应为 101，且 next_pid 应递增到 102。
  6. 两个进程并发执行时指令交错，可能出现两个进程都拿到 100 作为新 PID、next_pid 都只变成 101 的结果；无论多个线程的指令序列怎样交替执行程序都必须正常工作，不确定性要求并行程序的正确性，应先思考清楚问题、把程序的行为设计清楚，切忌急于编码再靠调试发现问题。
- **易错/考点**：该错误只在特定交错时序下出现，正常测试很难暴露，属于典型的竞态条件（race condition）；fork() 中"分配 PID"看似一行，实际是读-改-写三条以上指令。

9.2 一些概念part1（7.0 分钟）

- **主旨**：指出并发程序结果不确定的根源是 race condition（竞态条件）与"并非原子"的语句，并界定临界区、互斥、死锁、饥饿四个基本术语。
- **关键点**：
  1. Race Condition（竞态条件）指系统缺陷使结果依赖于并发执行或事件的顺序/时间，表现为不确定性和不可重现。

2. Atomic Operation（原子操作）指一次不存在任何中断或失败的执行：要么执行成功结束，要么根本没执行，并且不应被发现任何部分执行的状态。
3. 看上去原子的操作实际往往不原子：连 `x++` 这样的简单语句也由 3 条指令构成，pipeline、super-scalar、out-of-order、page fault 甚至会让单条机器指令都不原子。
4. 共享计数器例子中 A 执行 `while(i<10) i=i+1;` 、B 执行 `while(i>-10) i=i-1;` ，即使假定内存读取和存储是原子性的，"谁赢""是否一定有人赢"仍无保证。
5. Critical section（临界区）指进程中一段需要访问共享资源、并且当另一个进程处于相应代码区域时便不会被执行的那段代码区域。
6. Mutual exclusion（互斥）要求一个进程处于临界区访问共享资源时没有其他进程在临界区访问相同共享资源；Dead lock（死锁）是两个以上进程互相等待对方完成任务而都无法推进；Starvation（饥饿）是可执行进程被调度器持续忽略、虽可执行却不被执行。

- **易错/考点**：三个术语必须分清——互斥是"同时最多一个"、死锁是"互相等死"、饥饿是"被调度器跳过"；且"内存读写原子"推不出"`i=i+1` 原子"。

9.3 一些概念part2（12.6 分钟）

- **主旨**：用"去冰箱取面包"的生活类比，暴露朴素加锁方案在上下文切换下的漏洞，引出 lock（锁）与 waiting（等待）的语义。
- **关键点**：
  1. Person A 与 Person B 的时间线（3:00 各自查看冰箱、3:05 出门、3:15 到店、3:20/3:30 各自把面包放回冰箱）说明缺乏协调时两人会重复购买。
  2. "面包太多"问题的正确性质只有两条：最多有一个人去买面包；如果需要，一定有人去买面包。
  3. 在冰箱上加一把锁和钥匙（lockKey）可消除重复购买，却引出"有人想吃苹果也被挡住"的新问题，因此 lock（锁）的概念必须包含"等待"（waiting）。
  4. lock（锁）是在门、抽屉等物体上加保护性装置，使外人无法访问物体内的东西、只能等解锁后才能访问；deadlock（死锁）则是两人各持一把锁又互等对方那把，谁都无法继续执行。
  5. 便签方案 `if (!noteBread) { if (!noteA) { leave Note; buy bread; remove Note; } }` 因"检查便签"与"张贴便签"之间可能发生上下文切换而失效，两人都看到"没有便签"。
  6. 把 `leave Note` 提到最前面的"快速修复"虽然消除了重复购买，却导致"不会有人买面包"——用一个错误换掉了另一个错误。
- **易错/考点**：判断并发算法对错必须穷举"检查与修改之间被切换"的时机，语句前移只是把错误转移而非消除。

9.4 一些概念part3（11.3 分钟）

- **主旨**：用"带标签的便签"算法展示纯软件同步的复杂性与饥饿、忙等待代价，从而论证需要锁这一抽象。

- **关键点**：

  1. 加标签方案给两个线程各一张便签：Thread A 先 `leave note_1` 再 `if (no note_2)` 才买，Thread B 对称地先 `leave note_2` 再 `if (no note_1)`，避免双方同时看到空位。
  2. 该算法仍可能"没有线程去买面包"：错误时机的上下文切换会让每个线程都以为另一个线程会去买，这种状态被称为 starvation（饥饿）。
  3. 它有效但不够好：太复杂、难以说服自己它真的有效、A 和 B 的代码不同且每个线程代码可能都不同、线程过多时无法扩展。
  4. 线程在等待时一直在轮询并消耗 CPU 时间，这种情况叫 busy-waiting（忙等待）。
  5. 抽象方向是为每个线程保护一段 critical section（临界区）代码，由锁保证 mutual exclusion（互斥）：`Lock::Acquire()` 在锁被释放前一直等待然后获得锁，`Lock::Release()` 解锁并唤醒任何等待中的进程，二者必须是原子操作，否则两个线程可能同时发现锁已释放而都拿到锁。
  6. 面包问题于是简化为 `breadLock.Acquire(); if (noBread) buy bread; breadLock.Release();`，Acquire 与 Release 之间即"进入/退出临界区"。

- **易错/考点**：纯软件便签算法虽能互斥却会退化成"无人执行"或饥饿，且 Acquire/Release 若不保证原子，互斥本身就失效。

9.5 临界区（2.8 分钟）

- **主旨**：给出临界区方案必须满足的四条判据，并交代本章的技术路线。
- **关键点**：

  1. 互斥：同一时间临界区中最多存在一个线程。
  2. Progress：如果一个线程想要进入临界区，那么它最终会成功。
  3. 有限等待：如果一个线程 i 处于入口区，那么在 i 的请求被接受之前，其他线程进入临界区的时间是有限制的。
  4. 无忙等待（可选）：如果一个进程在等待进入临界区，那么在它可以进入之前会被挂起。
  5. 后续路线依次为：背景、一些概念、临界区、方法1 禁用硬件中断、方法2 基于软件的解决方法、方法3 更高级的抽象。

- **易错/考点**：四条判据中"无忙等待"是可选（性能）项，互斥、Progress、有限等待才是正确性必需。

9.6 方法1：禁用硬件中断（4.0 分钟）

- **主旨**：说明用关中断实现临界区互斥的机制、成立条件与代价。
- **关键点**：

  1. 进入和离开临界区的代码分别是 `ENTER_CRITICAL_SECTION` 与 `EXIT_CRITICAL_SECTION` 两条原语。

2. 基本机制有三类：禁用中断、软件方法（例如 Peterson 算法）、更高级的抽象；比较不同机制的维度是性能与并发级别。
3. 没有中断就没有上下文切换，因此没有并发：硬件将中断处理延迟到中断被启用之后，大多数现代计算机体系结构都提供指令来完成这一动作。
4. 具体做法即进入临界区时禁用中断、离开临界区时开启中断。
5. 一旦中断被禁用线程就无法被停止，整个系统都会为你停下来，并可能导致其他线程处于饥饿状态。
6. 若临界区可以任意长，就无法限制响应中断所需的时间（可能存在硬件影响），因此要小心使用。

- **易错/考点**：该方案把临界区长度与中断响应延迟绑在一起是主要风险，且只能用于单处理器（小结明确"禁用中断（仅限于单处理器）"）。

9.7 方法2：基于软件的解决方案（21.4 分钟）

- **主旨**：依次演示严格轮换、双标志、Peterson 与 Bakery 算法的失败与修正，说明纯软件同步的能力边界。
- **关键点**：

1. 线程通用结构为 `do { enter section 进入区域; critical section 临界区; exit section 离开区域; remainder section 剩余区域 } while (1);`，线程间共享一些变量来同步行为。
2. 严格轮换方案用 `int turn` 指示谁该进入：`while (turn != i);` 自旋、退出时 `turn = j`；它满足互斥但有时不满足 progress——对方在做别的事时你也必须等它用完临界区。
3. 双标志的两种顺序都失败：先等后置位（`while (flag[j]==1); flag[i]=1;`）没有互斥；先置位后等（`flag[i]=1; while (flag[j]==1);`）满足互斥但双方同时置位会死锁。
4. Peterson 算法（1981）用两个共享数据项 `int turn`（指示谁该进入临界区）和 `boolean flag[]`（指示进程是否准备好）：进入时 `flag[i]=TRUE; turn=j; while (flag[j] && turn==j);`，退出时 `flag[i]=FALSE;`，靠"意愿 + 轮次"打破对称。
5. Bakery 算法把方案推广到 N 个进程：进入临界区前进程接收一个数字，数字最小者先进；若 Pi 与 Pj 拿到相同数字则 i<j 时 Pi 先进，否则 Pj 先进；编号按枚举的增加顺序生成。
6. 小结：Dekker（1965）是第一个针对双线程的正确解、Bakery（Lamport 1979）解决 n 线程；它们都需要进程间的共享数据项、都需要忙等待而浪费 CPU 时间，且没有硬件保证就没有真正的软件解——Peterson 算法也需要原子的 LOAD 和 STORE 指令。

- **易错/考点**：交换两条语句的顺序就改变了错误类型（无互斥 ↔ 死锁），答题时必须说明"置位与检测之间被切换"的时序。

9.8 方法3：更高级的抽象（15.1 分钟）

- **主旨**：从硬件原子指令 Test-and-Set、Exchange 出发构造锁，并区分忙等待与无忙等待两种实现。
- **关键点**：

1. 层次关系：硬件提供中断禁用、原子操作指令等原语（大多数现代体系结构都这样），操作系统在其上提供锁、信号量等更高级编程抽象来简化并行编程。
2. 原子操作指令通过特殊的内存访问电路实现，针对单处理器和多处理器；`TestAndSet` 用 `rv = *target; *target = TRUE; return rv;` 返回旧值并把目标置忙，`Exchange` 原子地交换两个值，返回值让调用者知道"之前是否已被占用"。
3. 忙等待版锁：`class Lock { int value = 0; }`，`Acquire()` 为 `while (test-and-set(value)) ; //spin`，`Release()` 为 `value = 0`；锁空闲时置为忙并返回假，锁已忙时不改变状态并继续自旋。
4. 无忙等待版锁在 `class Lock` 中增加 `WaitQueue q`：`Acquire()` 失败时把当前 TCB 加入等待队列 q 并 `schedule()`，`Release()` 置 `value = 0`、从 q 中移除一个线程并 `wakeup()`。
5. 用 Exchange 也能实现锁：共享 `int lock = 0`，线程本地 `key = 1`，`while (key == 1) exchange(lock, key);` 进入临界区、`lock = 0` 退出。
6. 锁的缺点：忙等待消耗处理器时间；离开临界区且多个进程在等待时可能导致饥饿；低优先级进程持有临界区而高优先级进程抢占处理器时会死锁。三种实现为禁用中断（仅单处理器）、软件方法（复杂）、原子指令（单/多处理器均可）。

- **易错/考点**：自旋锁在锁已忙时并不改变锁状态、只是重复读取，别误判为"每次都在写"；且"禁用中断"这一实现只适用于单处理器。

第 10 章　信号量、管程与经典同步问题

> 接第 9 章的结论：信号量（P/V）与管程（条件变量）两种高层抽象。后半段用五个经典问题（生产者—消费者、读者—写者、哲学家进餐）反复演练，是全课最需要动笔推演的部分。

10.1 背景（2.5 分钟）

- **主旨**：说明并发程序为什么必须做同步，以及本章"硬件原语—高层抽象"的解决路线。
- **关键点**：
  1. 并发问题的根源是竞争条件（竞态条件）：多个程序并发执行、共享数据时，执行结果取决于指令的交叉顺序。
  2. 同步是多线程对共享公共数据的协调执行，包含互斥与条件同步两件事；互斥指同一时间只有一个线程能执行临界区。
  3. 保证同步正确很难，需要高层次的编程抽象（如锁），并最终由底层硬件支持、再编译实现。
  4. 层次图给出演进路线：多程序理念对应临界区，高层抽象对应锁，硬件支持则是禁用中断、原子操作（如 test-and-set）、原子 load/store。
  5. 锁的两种实现形态是阻塞（进入等待队列）与忙等待（spinlock），此外还有纯软件解决方式。
- **易错/考点**：互斥与条件同步是两个不同问题，锁本身只解决互斥；spinlock 属于忙等待，与"进入等待队列睡眠"要分清。

10.2 信号量（5.9 分钟）

- **主旨**：把同步需求抽象成一个整型变量加两个原子操作，即信号量及其 P/V 语义。
- **关键点**：
  1. 信号量是抽象数据类型：一个整型变量 sem 加上两个原子操作 P() 与 V()。
  2. P() 的语义：sem 减 1，若 sem<0 则等待（阻塞），否则继续执行。
  3. V() 的语义：sem 加 1，若 sem<=0 则唤醒一个正在等待的 P。
  4. 铁路类比：信号灯控制单轨区段，初始化为 2 个资源控制信号灯时，可允许两列车同时进入该区段。
  5. 由 Dijkstra 在 20 世纪 60 年代提出，P 取自荷兰语 "Probeer te Verlagen"（尝试减少），V 取自 "Verhoog"（增加）。
  6. 它是早期操作系统（如原 Unix）的主要同步原语，现在很少用，但在计算机科学研究中仍非常重要。
- **易错/考点**：P 与 V 的判断都在"修改之后"进行（减 1 后看是否 <0、加 1 后看是否 <=0），不要写成先判断后修改；P 会阻塞而 V 不会阻塞。

10.3 信号量的使用（20.8 分钟）

- **主旨**：讲清信号量既能做互斥又能做调度约束，并以有界缓冲区生产者—消费者问题贯穿示范。
- **关键点**：
  1. 信号量是受保护的变量：初始化完成后唯一能改变其值的途径是 P() 和 V()，且这两个操作必须原子执行。
  2. 假定信号量是"公平"的：不会有线程阻塞在 P() 上永不恢复（无饥饿），也不会被 V() 无限频繁地"插队"；实践中常采用 FIFO。
  3. 两类信号量：二进制信号量只取 0 或 1，一般（计数）信号量可取任意非负值，两者可以相互实现。
  4. 互斥用法：mutex = new Semaphore(1)，临界区前 mutex->P()、临界区后 mutex->V()。
  5. 调度约束用法：condition = new Semaphore(0)，线程 A 执行 condition->P() 等待，线程 B 执行 condition->V() 发出信号，即"P 等待、V 发出信号"。
  6. 有界缓冲区（mutex=1、fullBuffers=0、emptyBuffers=n）：Deposit 依次 emptyBuffers->P()、mutex->P()，放数据后 mutex->V()、fullBuffers->V()；Remove 依次 fullBuffers->P()、mutex->P()，取数据后 mutex->V()、emptyBuffers->V()。
- **易错/考点**：有界缓冲区要同时满足三条正确性要求（互斥、空则消费者等、满则生产者等），幻灯片末尾特意提问"P、V 操作的顺序有影响吗？"——互斥锁与资源信号量的先后次序决定程序是否还能正确推进。

10.4 信号量的实现（5.5 分钟）

- **主旨**：用硬件原语把 P/V 做成原子操作，并指出信号量编程本身的固有缺陷。
- **关键点**：
  1. 实现依赖硬件原语：禁用中断、原子指令 test-and-set，其思路与实现锁类似。
  2. 数据结构为 class Semaphore { int sem; WaitQueue q; }，即一个整型计数加一个等待队列。
  3. P() 的实现：sem--；若 sem<0，把当前线程 t 加入队列 q 并 block(t)。
  4. V() 的实现：sem++；若 sem<=0，从队列 q 中取出一个线程 t 并 wakeup(t)。
  5. 信号量有"双用途"（互斥与条件同步），但等待条件与互斥被耦合在同一机制里，读/开发代码比较困难，程序员必须非常精通信号量。
  6. 容易出错：使用的信号量已被另一个线程占用、忘记释放信号量；而且信号量本身不能够处理死锁问题。
- **易错/考点**：负的 sem 值代表还有多少线程在等待队列里，这正是 V 中用 sem<=0 判断"是否有人可唤醒"的原因；"信号量能解决死锁"是错误说法。

10.5 管程（24.9 分钟）

- **主旨**：用管程把互斥与条件同步的关注点分离，用条件变量的 Wait/Signal 表达"等某个条件成立"。
- **关键点**：
  1. 管程的目的就是分离互斥和条件同步的关注；它由一个锁（指定临界区）和 0 个或多个条件变量组成，条件变量用于等待/通知。
  2. 一般方法：把相关共享数据收集在对象/模块中，只定义方法来访问这些共享数据；线程从 entry queue 进入管程，条件变量 x、y 各有自己的等待队列与 urgent queue。
  3. Lock 的两个操作：Acquire() 等待直到锁可用再抢占，Release() 释放锁并在有等待者时唤醒。
  4. 条件变量允许线程在临界区内进入等待状态：Wait() 释放锁、睡眠，被唤醒后重新获得锁才返回；Signal()（或 broadcast()）唤醒一个（或所有）等待者。
  5. 实现要点：Condition 内含 numWaiting 计数与 WaitQueue q；Wait 中依次 numWaiting++、入队、release(lock)、schedule()、重新 require(lock)；Signal 中若有等待者则出队、wakeup(t)、numWaiting--。
  6. 两种语义：Hansen-style（多数真实 OS、Java、Mesa）中 Signal 只是"提示"，条件可能已不成立，必须用 while 重新检查；Hoare-style（多数教材）语义更清晰、便于证明，但实现代价高。两种 Deposit 的差别正是 while 与 if。
- **易错/考点**：条件变量必须与锁配对使用，且判断条件要用 while 循环重检（Mesa/Hansen 语义）而不是 if，否则会漏掉唤醒导致条件判断失效。

10.6 经典同步问题-1（10.8 分钟）

- **主旨**：用信号量解决读者—写者问题，并说明读者优先/写者优先各自带来的饥饿。
- **关键点**：
  1. 问题的约束：允许同一时间有多个读者，但任何时候只有一个写者；没有写者时读者才能访问数据，没有读者和写者时写者才能访问，任何时候只能有一个线程操作共享变量。
  2. 共享数据包括数据集本身，加上信号量 CountMutex（初值 1）、WriteMutex（初值 1）和整数 Rcount（初值 0）。
  3. 写者最简单：sem_wait(WriteMutex)、write、sem_post(WriteMutex)，天然互斥。
  4. 读者只在 Rcount==0 时先 sem_wait(WriteMutex)，然后 ++Rcount 进入读；读完 --Rcount，若减到 0 才 sem_post(WriteMutex)。
  5. 多个读者会并发修改 Rcount，因此计数段必须用 CountMutex 包住：进入前 sem_wait(CountMutex)、改完 sem_post(CountMutex)，退出时再重复一次。
  6. 读者优先策略下只要有读者活跃，后来的读者都被接纳，读者源源不断时写者始终阻塞；写者优先则一旦写者就绪就尽快写，写者源源不断时读者始终阻塞（如何实现留作思考）。

- **易错/考点**：Rcount 的增减本身就是临界区，漏掉 CountMutex 会破坏"最后一个读者才释放 WriteMutex"的判断，出现读者与写者同时进入。

10.7 经典同步问题-2（5.3 分钟）

- **主旨**：用 monitor（管程）而非信号量手工配对的方式，重新组织读者—写者问题的同步逻辑。
- **关键点**：
  1. 基本结构只有两个方法：`Database::Read()` 与 `Database::Write()`，各自以「等待条件 + 读/写 + 检查离开时唤醒对方」三段式书写。
  2. `Read()` 开头 `Wait until no writers;`，进入后执行 `read database;`，离开时 `check out – wake up waiting writers;`。
  3. `Write()` 开头 `Wait until no readers/writers;`，写入后 `check out – wake up waiting readers/writers;`，即写者离开要同时唤醒两类等待者。
  4. 管程内部状态变量：`AR`（活跃读者数）、`AW`（活跃写者数）、`WR`（等待读者数）、`WW`（等待写者数），全部是计数器而非单纯标志位。
  5. 两个条件变量 `Condition okToRead`、`Condition okToWrite` 配合一把 `Lock lock`，锁只保护状态变量的互斥访问，等待与唤醒靠条件变量完成。
- **易错/考点**：管程把「互斥」与「同步」拆成 lock 与 condition 两套机制，别再把 P/V 直接套进管程代码里。

10.8 经典同步问题-3（15.8 分钟）

- **主旨**：把管程版读者—写者方案落到可执行的 StartRead/EndRead/StartWrite/EndWrite 四个过程上，说明判断条件为何必须用 while。
- **关键点**：
  1. 公有过程 `Database::Read()` 依次调用 `StartRead(); read database; DoneRead();`，`Write()` 调用 `StartWrite(); write database; DoneWrite();`。
  2. `StartRead`：`lock.Acquire();` 后 `while (WW > 0) okToRead.wait(&lock);` 再 `AR++; lock.Release();` ——有写者排队时读者必须让路，避免写者饿死。
  3. `DoneRead`：`lock.Acquire(); AR--; if (AR == 0 && WW > 0) okToWrite.signal(); lock.Release();`，只有最后一个读者离开才唤醒写者。
  4. `StartWrite`：`lock.Acquire(); while ((AR > 0) || (WW > 0)) okToWrite.wait(&lock); WW--; AW++; lock.Release();`，写者之间也要互斥。
  5. `DoneWrite`：`lock.Acquire(); AW--; if (WW > 0) okToWrite.signal(); else if (WR > 0) okToRead.broadcast(); lock.Release();` ——写者优先唤醒写者，无写者时才广播唤醒全部读者。

6. 条件判断用 while 而非 if，是因为被 signal 唤醒后条件可能已被其他进程破坏，必须重新检查。

- **易错/考点**：`AR` / `AW` / `WR` / `WW` 四个计数器的增减时机与 `signal` / `broadcast` 的选择（写者用 signal 逐个唤醒，读者用 broadcast 集体唤醒）。

10.9 经典同步问题-4（13.7 分钟）

- **主旨**：引入 1965 年 Dijkstra 提出的哲学家进餐问题，并用四种由简到繁的方案展示死锁的产生与规避。
- **关键点**：

1. 问题描述：5 个哲学家围坐圆桌，两人之间放 1 把叉子共 5 把，哲学家交替「思考」与「进餐」，进餐需同时拿起左右两把叉子。
2. 共享数据为 `Bowl of rice (data set)` 与 `Semaphore fork[5] initialized to 1`，取放叉子即 `take_fork(i): P(fork[i])`、`put_fork(i): V(fork[i])`。
3. 方案一按固定顺序先左后右取叉，代码注释直接判定「不正确，可能导致死锁」——5 人同时各拿左叉即形成循环等待。
4. 方案二改为奇数号哲学家先取右叉、偶数号先取左叉（`if (fork((i+1)%N))` 分支），画面结论是「对拿叉子的过程进行了改进，但仍不正确」。
5. 方案三在拿不到右叉时 `put_fork(i)` 放回左叉并 `wait_some_time(i)` 随机等待，「等待时间随机变化。可行，但非万全之策」，属于工程权宜而非理论保证。
6. 方案四用 `semaphore mutex` 把取叉到放叉整段包成临界区，「互斥访问。正确，但每次只允许一人进餐」；缺陷是把叉子当临界资源，5 把叉本应允许两个不相邻哲学家同时进餐，造成资源浪费。

- **易错/考点**：破坏死锁的四种思路（破坏请求保持、破坏循环等待、加时间戳/随机退避）分别对应上述哪个方案。

10.10 经典同步问题-5（16.5 分钟）

- **主旨**：跳出「抢叉子」的直觉，改以状态机加信号量的方式，让哲学家只在左右邻居都不进餐时才拿起两把叉子。
- **关键点**：

1. 思路一（人的角度）：指导原则是「要么不拿，要么拿两把叉子」，把行为拆成 S1 思考中、S2 进入饥饿状态、S3 若左邻或右邻正在进餐则阻塞、S4 拿起两把叉子、S5 吃面条、S6 放下左叉并看右邻能否进餐、S7 放下右叉并看左邻、S8 转 S1 的循环。
2. 思路二（计算机角度）：指导原则是「不能浪费 CPU 时间；进程间相互通信」，把上述流程翻译为带阻塞/唤醒的状态转移，S3 进入阻塞态、S6/S7 在邻座处于饥饿且两叉齐备时唤醒对方。
3. 思路三（编程角度）三条要求：需要数据结构描述每个哲学家当前状态；该状态是临界资源，访问必须互斥；吃饱后要唤醒左邻右舍，存在同步关系。

4. 数据结构：`#define N 5`、`#define LEFT (i-1)%N`、`#define RIGHT (i+1)%N`，状态常量 `THINKING 0`、`HUNGRY 1`、`EATING 2`，以及 `int state[N]`、`semaphore mutex`（互斥信号量，初值 1）、`semaphore s[N]`（同步信号量，初值 0）。
5. `take_forks(i)`：`P(mutex); state[i]=HUNGRY; test_take_left_right_forks(i); V(mutex); P(s[i]);` ——先在自己进程内标记饥饿并测试，再在管程外阻塞等待，避免持锁睡眠。
6. `test_take_left_right_forks(i)`：条件为 `state[i]==HUNGRY && state[LEFT]!=EATING && state[RIGHT]!=EATING`，成立则置 `state[i]=EATING` 并 `V(s[i])` 通知第 i 人可以吃饭。

- **易错/考点**：`P(s[i])` 放在 `V(mutex)` 之后是关键——若放在临界区内会持锁阻塞导致全系统死锁。

10.11 景点同步问题-6（11.6 分钟）

- **主旨**：补全 put_forks 与哲学家主循环，说明进餐结束如何级联唤醒左右邻居，收束整个信号量解法。
- **关键点**：

1. `put_forks(i)`：`P(mutex); state[i]=THINKING; test_take_left_right_forks(LEFT); test_take_left_right_forks(RIGHT); V(mutex);` ——放下叉子即恢复思考态，并在同一临界区内替左右邻居各做一次测试。
2. 两次 `test_take_left_right_forks` 分别针对 `LEFT` 与 `RIGHT`，只有邻居处于 HUNGRY 且两侧都不在 EATING 时才会被唤醒，实现「吃饱后唤醒左邻右舍」的同步关系。
3. 唤醒动作通过 `V(s[i])` 完成，被唤醒者在 `take_forks` 的 `P(s[i])` 处继续，随即进入 eat()。
4. 哲学家主循环 `philosopher(int i)` 为 `while(TRUE){ think(); take_forks(i); eat(); put_forks(i); }`，S1、S2~S4、S5、S6~S7 四段与前述状态机一一对应。
5. 结尾提出 `think()`、`eat()` 是否需要进一步实现的问题，即这两个函数在本问题中只作为占位，不涉及具体业务。

- **易错/考点**：画面以代码为主；`test_take_left_right_forks` 既被自己调用也被邻居调用，是「状态驱动 + 被动唤醒」的核心，注意它不是让调用者去吃，而是检查参数 i 能否吃。

第 11 章　死锁与进程间通信

> 死锁的四个必要条件、预防/避免/检测三条路线，重点是银行家算法的安全性检查。后半章转向 IPC：共享内存、消息队列、管道、信号四类机制各自解决什么问题。

11.1 死锁问题（3.5 分钟）

- **主旨**：用单车道桥的类比说明什么是死锁，并给出死锁的形式化定义。
- **关键点**：
  1. 单车道桥模型中流量只在一个方向，桥的每个部分都可以看作一个资源，每辆车相当于一个进程。
  2. 若发生死锁，可能只需一辆车倒退即可解决，对应抢占资源和回滚（rollback）。
  3. 若死锁严重，可能几辆车都必须倒退，代价随参与进程数增加。
  4. 车一直退让可能使某辆车长期得不到通行，即可能发生饥饿（starvation）。
  5. 死锁的定义：一组阻塞的进程持有一种资源并等待获取另一个进程所占有的一种资源。
  6. 典型例子：系统只有 2 个磁带驱动器，P1 和 P2 各持有一个，又都需要另外一个，于是互相等待。
- **易错/考点**：死锁与饥饿不同——死锁的进程集合互相循环等待、永远无法推进，饥饿只是某个进程长期被跳过；桥的例子中"一辆车倒退"对应资源抢占。

11.2 系统模型（20.4 分钟）

- **主旨**：建立资源、资源实例与资源分配图（resource allocation graph）这套描述死锁的形式化模型。
- **关键点**：
  1. 资源类型记为 R1, R2, ..., Rm，如 CPU cycles、memory space、I/O devices；每个资源类型 Ri 有 Wi 个实例（instance）。
  2. 进程使用资源的三个规范动作：request/get（申请，空闲才可分配）、use/hold（占用）、release（释放回空闲池）。
  3. 可重用资源（reusable resource）：同一时间只能被一个进程使用且不能被删除，进程获得后释放再由其他进程重用，如处理器、I/O 通道、主存和辅存、文件、数据库、信号量。
  4. 消耗型资源（consumable resource）：可被创建和销毁，如 I/O 缓冲区中的中断、信号、消息、信息；接收消息被阻塞时可能死锁。
  5. 资源分配图是一组顶点 V 和边 E 的集合，顶点分两类：进程集合 P = {P1, ..., Pn} 与资源集合 R = {R1, ..., Rm}。
  6. 申请边（requesting/claiming edge）为有向边 Pi → Rj，分配边（assignment/holding edge）为有向边 Rj → Pi，图中资源框内的小圆点表示实例数。

- **易错/考点**：申请边方向是"进程指向资源"，分配边方向是"资源指向进程"，画反是常见错误；图中没有环一定无死锁，但有环不一定死锁——若环上每个资源类型都只有一个实例则必死锁，多实例时环只是必要条件（画面以示意图讲解为主）。

11.3 死锁特征（5.3 分钟）

- **主旨**：给出死锁发生的四个必要条件（necessary conditions），并用资源分配图对比死锁与无死锁状态。
- **关键点**：
  1. 互斥（mutual exclusion）：在一个时间只能有一个进程使用资源。
  2. 持有并等待（hold and wait）：进程保持至少一个资源，同时正在等待获取其他进程持有的额外资源。
  3. 无抢占（no preemption）：资源只能被持有它的进程自愿释放，即进程完成任务之后才释放。
  4. 循环等待（circular wait）：存在等待进程集合 {P0, P1, ..., Pn}，P0 等待 P1 占用的资源，P1 等待 P2 占用的资源，...，Pn 等待 P0 占用的资源。
  5. 四个条件必须同时成立死锁才可能出现，故称必要条件，破坏其中任意一个即可防止死锁。
  6. 幻灯片用两张资源分配图对比：左侧图成环且资源实例不足导致死锁，右侧图无环故没有死锁。
- **易错/考点**：四个条件"同时成立"只是必要条件而非充分条件，答题时不要写成"满足四个条件一定死锁"。

11.4 死锁处理办法（3.6 分钟）

- **主旨**：梳理操作系统处理死锁的三条总体路线，并说明实际系统通常选择忽略。
- **关键点**：
  1. 路线一：确保系统永远不会进入死锁状态，对应死锁预防（Deadlock Prevention）与死锁避免（Deadlock Avoidance）。
  2. 路线二：允许系统进入死锁状态，然后用死锁检测（Deadlock Detection）和死锁恢复（Recovery from Deadlock）事后处理。
  3. 路线三：忽略这个问题，假装系统中从来没有发生死锁，用于大多数操作系统，包括 UNIX。
  4. 选择忽略的理由是死锁发生概率低，而预防/避免/检测的开销要持续付出。
- **易错/考点**：预防（prevention）是静态地破坏必要条件，避免（avoidance）是运行时动态判断分配是否安全，二者不要混为一谈。

11.5 死锁预防和死锁避免（15.4 分钟）

- **主旨**：分别给出破坏四个必要条件的预防手段，以及基于安全状态判断的避免策略。

- **关键点**：

  1. 破坏互斥：共享资源不是必须的，但必须占用非共享资源，故互斥一般无法破坏。
  2. 破坏持有并等待：要求进程在开始执行前一次性请求并分配到所有资源，或允许申请新资源仅当它当前没有持有任何资源；代价是资源利用率低、可能发生饥饿。
  3. 破坏无抢占：若进程已占有某些资源又申请不能立即分配的资源，则释放它当前占有的资源；被抢占的资源加入被阻塞进程的列表，只有当它能同时获得旧资源和新申请的资源时才可执行。
  4. 破坏循环等待：对所有资源类型进行排序，要求每个进程严格按资源编号递增的顺序申请。
  5. 死锁避免需要额外先验信息：最简单有效的模式是要求每个进程声明它可能需要的每类资源的最大数目。
  6. 安全状态定义：存在序列 ，对每个 Pi，其需求能由当前可用资源加上所有 Pj（j < i）已持有的资源满足；Pi 完成即释放并归还资源，依次类推，全序列可完成则系统安全。
  7. 资源分配图算法：claim edge（声明边）Pi → Rj 用虚线表示，进程请求时转为 request edge（申请边），资源释放后 assignment edge 转回 claim edge，成环即不安全。

- **易错/考点**：安全状态一定不死锁，不安全状态只是"可能"死锁；银行家算法属于避免而非预防，且必须事先知道最大需求。

11.6 银行家算法（22.9 分钟）

- **主旨**：以银行放贷类比资源分配，用安全性检查动态决定是否满足进程的资源请求。
- **关键点**：

  1. 背景：由艾兹格·迪杰斯特拉于 1965 年为 T.H.E 系统设计；客户须在第一次贷款时声明所需最大资金量，银行家在不超过自身持有量时尽量满足，客户完成项目后归还。
  2. 前提条件：每类资源有多个实例、每个进程必须能最大限度地利用资源、请求不到就等待、获得全部资源后必须在有限时间内释放。
  3. 数据结构：n 为进程数、m 为资源类型数；Max（总需求量）n×m、Available（剩余空闲量）长度 m、Allocation（已分配量）n×m、Need（未来需要量）n×m。
  4. 核心恒等式：Need[i, j] = Max[i, j] − Allocation[i, j]。
  5. 安全性算法（Safety State Estimating Algorithm）：初始化 Work = Available、Finish[i] = false；反复寻找满足 Finish[i] = false 且 Need_i ≤ Work 的 i，令 Work = Work + Allocation_i、Finish[i] = true；若最终所有 Finish[i] = true 则处于安全状态。
  6. 请求处理三步：先判 Request_i ≤ Need_i（否则说明超过最大需求，报错），再判 Request_i ≤ Available（否则 Pi 必须等待），然后试探性分配 Available −= Request_i、Allocation_i += Request_i、Need_i −= Request_i，再跑安全性检查，不安全则回滚。
  7. 实例：5 个进程 P0–P4、3 类资源 R1–R3，资源向量 R = (10, 5, 7)、Available V = (3, 3, 2)，演示 P1 运行完成、P4 运行完成的安全序列，以及 P1 再申请 R1、R3 各一个后转为不安全状态。

- **易错/考点**：判断 Request_i ≤ Need_i 用的是 Need 而非 Max；试探分配后必须做安全性检查并支持回滚，顺序颠倒或漏掉回滚是失分点。

11.7 死锁检测和死锁恢复（14.8 分钟）

- **主旨**：在允许死锁发生的策略下，如何检测死锁（含单实例与多实例两种算法）以及检测后如何恢复。
- **关键点**：

  1. 每个资源类型只有单一实例时，维护 wait-for graph（等待图）：节点是进程，Pi → Pj 表示 Pi 等待 Pj 占用的资源，定期检测图中是否存在环，算法需 $O(n^2)$ 次操作。
  2. 多实例时用死锁检测算法：初始化 Work = Available，Allocation_i ≠ 0 的进程 Finish[i] = false、否则为 true；反复找 Finish[i] = false 且 Request_i ≤ Work 的 i，令 Work = Work + Allocation_i、Finish[i] = true。
  3. 检测算法结束时若存在 Finish[i] = false，则系统处于死锁状态，且该 Pi 就是死锁进程；算法需要 $O(m \times n^2)$ 次操作。
  4. 实例：5 个进程 P0–P4、资源 A(7)、B(2)、C(6)，给出 T0 时刻的 Allocation、Request、Available 表，存在序列 使所有 Finish[i] = true，故此刻无死锁。
  5. 检测的时机与频率取决于：死锁多久会发生、多少进程需要被回滚；如果算法被多次调用，资源图可能有多个环，难以分辨哪些进程"造成"了死锁。
  6. 恢复手段之一为进程终止：可终止所有死锁进程，也可按时间片逐个终止直到死锁消除；终止顺序依据进程优先级、已运行时间与剩余时间、占用资源、完成任务还需的资源、需终止的进程数、是交互式还是批处理。
  7. 恢复手段之二为资源抢占：选择成本最小的受害者（victim），回滚到某个安全状态后重启进程；同一进程可能反复被选为受害者而产生饥饿（含回滚次数的影响）。

- **易错/考点**：wait-for graph 只适用于每类资源单实例，多实例必须用类似银行家算法的矩阵检测；检测算法中的向量是 Request 而非 Need。

11.8 IPC概述（8.0 分钟）

- **主旨**：说明进程间通信（IPC）的基本模型、命名方式与同步语义。
- **关键点**：

  1. 进程通信的机制及同步针对的是不使用共享变量的进程通信，IPC facility 提供两个基本操作：send(message) 与 receive(message)，消息大小可固定也可变。
  2. 若 P 和 Q 要通信，需要在其间建立通信链路，并通过 send/receive 交换消息。
  3. 通信链路的实现分物理的（如共享内存、硬件总线）和逻辑的（如逻辑属性）。
  4. 直接通信：进程必须正确命名对方，send(P, message) 发送到进程 P，receive(Q, message) 从进程 Q 接收。

5. 直接通信链路的属性：自动建立链路、一条链路恰好对应一对通信进程、每对进程之间只有一条链路、链路可单向但通常为双向。
6. 间接通信：消息发往/取自消息队列（mailbox），每个队列有唯一 ID，只有共享同一队列的进程才能通信；原语为 send(A, message)、receive(A, message)。
7. 间接通信的链路属性：只有共享同一消息队列才建立链路、一条链路可与多个进程关联、每对进程可共享多条链路、连接可单向或双向；操作包括创建、发送/接收、销毁消息队列。
8. 阻塞发送/接收是同步的（发送方阻塞到消息被接收、接收方阻塞到消息可用），非阻塞是异步的（发送后继续、接收返回有效消息或 null）。

- **易错/考点**：直接通信以"进程"为地址，间接通信以"消息队列"为地址；阻塞/非阻塞描述同步语义，与缓冲区容量是两组独立维度。

11.9 信号、管道、消息队列和共享内存（22.7 分钟）

- **主旨**：介绍四类具体 IPC 机制的实现方式、优缺点与同步问题。
- **关键点**：

1. 链路缓冲（queue）的三种容量：0 容量即无缓冲，发送方必须等待接收方（rendezvous，汇合）；有限容量 n 时队列满则发送方等待；无限容量时发送方从不需要等待。
2. 信号（signal）是软件中断式的事件通知，例子有 SIGFPE、SIGKILL、SIGUSR1、SIGSTOP、SIGCONT。
3. 收到信号后的三种处置：catch（调用用户指定的处理函数）、ignore（依赖系统默认操作，如内存转储、挂起或恢复进程）、mask（屏蔽，屏蔽的信号不会被递送）；也可由硬件产生，如处理器陷阱类信号。
4. 信号的不足：不能传输要交换的任何数据，只能传递"发生了某事件"。
5. 管道（pipe）：子进程从父进程继承文件描述符 0 stdin、1 stdout、2 stderr，进程无需知道（或不关心）数据实际来自键盘、文件还是网络；示例 % ls | more 中前一个进程把 stdout 设为管道写端，后一个进程把 stdin 设为管道读端。
6. 消息队列以 FIFO 方式管理消息，message 作为字节序列存储，本质是消息数组，多个生产者 send、接收方 receive 到队列。
7. 共享内存：每个进程有私有地址空间，在其中创建共享内存段映射到同一物理页框，优点是快速、方便地共享数据，不足是必须由程序员同步数据访问。
8. 共享内存是最快的 IPC：一个进程写入另一进程立即可见，没有系统调用干预、没有数据复制，但不同步。

- **易错/考点**：信号不能携带数据、共享内存不提供同步，是两个高频考点；管道是单向字节流且依赖 fork 继承的文件描述符，匿名管道不能用于无亲缘关系的进程。

第 12 章　文件系统

> 从「持久存储的抽象」讲起，一路向下：文件与目录、VFS 抽象层、打开文件的数据结构，再到磁盘上的块分配（连续/链式/索引）、空闲空间管理、RAID 与磁盘调度。本章概念多但难度平缓，适合配合实验一起看。

12.1 文件系统：总体介绍（1.7 分钟）

- **主旨**：给出文件系统这一章的知识地图，说明从抽象概念到磁盘布局、再到性能与可靠性的讲解顺序。
- **关键点**：
  1. 本章目录依次为：基本概念、虚拟文件系统、数据块缓存、打开文件的数据结构、文件分配、空闲空间列表、多磁盘管理 - RAID、磁盘调度。
  2. 前两部分属于抽象层：先定义文件与目录等基本概念，再用虚拟文件系统（VFS）屏蔽不同文件系统的差异。
  3. 数据块缓存与打开文件的数据结构都在内存一侧：前者缓存磁盘块以减少 I/O，后者记录进程当前打开的文件。
  4. 文件分配、空闲空间列表属于磁盘上的空间管理，决定一个文件的数据块如何分布以及如何记录未用空间。
  5. 多磁盘管理 - RAID 面向多块磁盘的组织与冗余，磁盘调度决定磁盘请求的响应次序。
- **易错/考点**：这张目录本身就是考点，要能说出八个部分分别属于"抽象接口—运行时结构—磁盘空间管理—性能可靠性"哪一层。

12.2 基本概念（2.0 分钟）

- **主旨**：交代基本概念一节内部的组织顺序，为后面逐条展开作铺垫。
- **关键点**：
  1. 基本概念分为五部分：文件系统和文件、文件描述符、目录、文件别名、文件系统种类。
  2. "文件系统和文件"先给出两个核心对象的定义，是后面所有内容的语义基础。
  3. "文件描述符"讲的是进程运行期如何引用一个已打开的文件，属于内存中的动态视图。
  4. "目录"与"文件别名"都属于命名问题：前者解决名字到文件的查找，后者解决一个文件多个名字。
  5. 最后落到"文件系统种类"，说明上述概念在不同类型文件系统中的不同实现方式。
- **易错/考点**：注意五部分的排列逻辑是由静态对象（文件）到动态引用（描述符）再到命名（目录、别名），不要记成随意罗列。

12.3 基本概念——文件系统和文件（3.7 分钟）

- **主旨**：给出文件系统与文件这两个最基本对象的定义，说明文件系统是操作系统对持久存储的抽象。
- **关键点**：
  1. 文件系统是"一种用于持久性存储的系统抽象"。
  2. 它在存储器上完成组织、控制、导航、访问和检索数据的工作。
  3. 大多数计算机系统都包含文件系统：个人电脑、服务器、笔记本电脑。
  4. 嵌入式与消费电子设备同样有文件系统：iPod、Tivo/机顶盒、手机/掌上电脑。
  5. 规模可以极大：Google 可能就是由一个文件系统构成的。
  6. 文件是"文件系统中一个单元的相关数据在操作系统中的抽象"。
- **易错/考点**：文件系统是持久存储的抽象、文件是其中的一个单元，二者是整体与单元的关系，不能混为一谈。

12.4 基本概念——文件系统的功能（2.7 分钟）

- **主旨**：把文件系统要做的事归纳为空间分配、集合管理、便利与保护三大类。
- **关键点**：
  1. 分配文件磁盘空间包含三件事：管理文件块（哪一块属于哪一个文件）、管理空闲空间（哪一块是空闲的）、分配算法（策略）。
  2. 管理文件集合首先是定位文件及其内容。
  3. 命名是"通过名字找到文件的接口"，最常见的形式是分层文件系统。
  4. 文件系统类型即"组织文件的不同方式"，同类功能可以有不同实现。
  5. 提供的便利及特征包括保护——用分层来保护数据安全。
  6. 可靠性/持久性要求即使发生崩溃、媒体错误、攻击等，文件仍能保持持久。
- **易错/考点**：块管理与空闲空间管理是数据结构层面的问题，分配算法是策略层面的问题，答题时不要混为一类。

12.5 基本概念——文件和块（0.9 分钟）

- **主旨**：说明文件的元数据组成，以及文件头如何把逻辑文件映射到物理块。
- **关键点**：
  1. 文件属性包括名称、类型、位置、大小、保护、创建者、创建时间、最近修改时间等。
  2. 文件头在存储元数据中保存了每个文件的信息。
  3. 文件头保存文件的属性。

4. 文件头还跟踪哪一块存储块属于逻辑上文件结构的哪个偏移。

- **易错/考点**：文件头（file header，即 inode 一类结构）存的是元数据与块到逻辑偏移的映射，本身不含文件内容。

12.6 基本概念——文件描述符（15.8 分钟）

- **主旨**：讲清进程如何通过打开文件表和文件描述符访问文件，并说明字节视图与块视图的差别、访问模式与共享语义。
- **关键点**：

1. 使用文件必须先用 open 打开，典型调用序列为 f = open(name, flag) → read(f, ...) → close(f)；操作系统为每个进程维护一张打开文件表（open file table），文件描述符（file descriptor）就是这张表中的索引。
2. 管理打开文件需要元数据：文件指针（指向最近一次读写位置，每个打开该文件的进程都有）、文件打开计数（最后一个进程关闭时才允许从打开文件表移除）、文件磁盘位置（缓存数据访问信息）、访问权限（每个程序的访问模式）。
3. 用户视图是"字节的集合"（UNIX），系统不关心磁盘上的任何数据结构；操作系统内部视角是"块的集合"，块是逻辑转换单元、扇区是物理转换单元，UNIX 中块大小为 4KB。
4. 读写都以整块为单位：读 2^12 字节需先获取字节所在的块、再返回块内对应部分；写则取块—修改块内对应部分—写回块，所以 read()/write() 即使每次只访问 1 字节也会读写整个块（画面部分文字被遮挡）。
5. 访问模式分顺序访问（按字节依次读取，几乎所有访问都是这种方式）、随机访问（从中间读写，虚拟内存支持文件即内存页存储在文件中时会用到）、索引访问（通过特征）；文件内部结构分无结构（单词、比特的队列）、简单记录结构（列、固定长度/可变长度）、复杂结构（如 MS Word、PDF、可执行文件）。
6. 共享与保护：UFS 语义下写入内容立即对其他打开同一文件的用户可见且共享文件指针允许多用户同时读写，会话语义下写入内容只有文件关闭时才可见，另有锁机制；保护手段有访问控制列表 ACL（⟨文件实体，权限⟩）和 Unix 模式（⟨用户|组|所有人，读|写|可执行⟩，用户 ID 与组 ID 决定权限）。

- **易错/考点**：文件描述符只是打开文件表的索引而不是文件本身；块大小≠扇区大小；访问 1 字节也会引发整块 I/O，这是理解文件缓存与性能的关键。

12.7 基本概念——目录（6.1 分钟）

- **主旨**：说明目录如何组织文件、目录项的结构与实现方式，以及名字解析与挂载机制。
- **关键点**：

1. 文件以目录的方式组织起来，目录本身是一种特殊的文件，每个目录项包含一个字符串和一个指针（name, pointer to file header），形成目录与文件的树型结构。

2. 早期文件系统是单层目录（只有一张目录），现代系统使用层次名称空间，如 /usr/lib 下的 spell、mail、passwd、programs 等。
3. 典型操作包括：搜索文件、创建文件、删除文件、枚举目录、重命名文件、在文件系统中遍历一个路径；其中目录的修改只允许内核模式完成以确保映射的完整性，而读目录（如 ls）允许应用程序进行。
4. 目录实现有两种：文件名的线性列表（含指向数据块的指针，编程简单但执行耗时）与 hash 表（hash 数据结构的线性表，减少目录搜索时间，但存在两个文件名 hash 值相同的碰撞问题且大小固定）。
5. 名字解析是把逻辑名字转换成物理资源（如文件）的过程，即遍历文件目录直到找到目标文件；解析 "/bin/ls" 需读取 root 的文件头（在磁盘固定位置）→读 root 数据块搜索 "bin" 项→读 bin 的文件头→读 bin 数据块搜索 "ls" 项→读 ls 的文件头。
6. 每个进程都有一个当前工作目录用于解析文件名，允许用相对路径代替绝对路径；文件系统必须先挂载才能被访问，未挂载的文件系统被挂载在挂载点上。

- **易错/考点**：路径解析必须逐级读目录项、不能直接跳到目标；目录的修改限于内核模式而读目录可在用户态，这一区分常被考到。

12.8 基本概念——文件别名（5.9 分钟）

- **主旨**：解释一个文件对应多个名字（链接）的实现方式，以及由此带来的悬空指针与循环问题。
- **关键点**：

1. 两个或多个文件名可以关联同一个文件，如 /dict/count 与 /spell/count、/dict/w/list 与 /spell/words/list 指向同一份数据。
2. 硬链接是多个文件项指向一个文件；软链接是以"快捷方式"指向其他文件，通过存储真实文件的逻辑名称来实现。
3. 删除一个有别名的文件时，该别名将成为一个"悬空指针"。
4. Backpointers 方案：每个文件维护一个包含多个 backpointers 的列表，删除时据此删除所有反向指针，这些 backpointers 使用菊花链管理。
5. 另一种做法是添加一个间接层——目录项数据结构，链接表示"已存在文件的另外一个名字（指针）"，链接处理即跟随指针来定位文件。
6. 为保证不出现循环：只允许到文件的链接、不允许在子目录的链接，每增加一个新链接都用循环检测算法确定是否合理；实践中还限制路径可遍历文件目录的数量。

- **易错/考点**：硬链接指向文件本体、软链接存的是目标路径字符串；删除最后一个硬链接才真正释放文件，软链接的目标被删后留下悬空指针。

12.9 基本概念——文件系统种类（7.8 分钟）

- **主旨**：按存储介质与访问方式给文件系统分类，并指出网络/分布式共享文件带来的新问题。
- **关键点**：
  1. 磁盘文件系统把文件存放在磁盘等存储设备上，代表有 FAT、NTFS、ext2/3、ISO9660 等。
  2. 数据库文件系统按数据特性或属性（如 B 树、哈希）对文件数据块寻址，代表如 MySQL。
  3. 日志文件系统记录文件系统的修改事件，代表如 Journaling File System，可据日志恢复一致性。
  4. 网络/分布式文件系统中文件位于远程服务器，客户端远程挂载服务器文件系统，标准文件系统访问协议被转换为进程访问。
  5. 标准文件共享协议：NFS 用于 Unix、CIFS 用于 Windows。
  6. 分布式文件系统的问题：客户端与服务器上的用户标识映射很复杂、NFS 不安全、一致性问题、错误处理模式。
- **易错/考点**：网络文件系统与分布式文件系统常被混为一谈，NFS 属于远程挂载而非本地磁盘格式；另有「特殊/虚拟文件系统」一类，画面未展开。

12.10 虚拟文件系统（9.1 分钟）

- **主旨**：在文件系统 API 与具体文件系统之间插入 VFS（虚拟文件系统）抽象层，使上层用同一套接口访问 ext2/fat/iso9660/nfs/smb。
- **关键点**：
  1. 分层结构：上层是虚拟（逻辑）文件系统，下层是特定文件系统模块，中间为虚拟文件系统层，其下接 Device I/O 与 Network I/O。
  2. 目的：对所有不同文件系统做统一抽象；功能是提供相同的文件和文件系统接口、管理文件与文件系统关联的数据结构、高效查询遍历文件系统、与特定文件系统模块交互。
  3. 卷控制块（Unix superblock）每个文件系统一个，记录块大小、空闲块、计数/指针等文件系统级详细信息。
  4. 文件控制块（Unix inode）每个文件一个，记录许可、拥有者、大小、数据块位置等文件级详细信息。
  5. 目录节点（Unix dentry）每个目录项一个，把目录项名称映射到对应控制块，并由父子节点、项目列表构成树状结构。
  6. 三类结构持久存放在二级存储的数据块中，运行时按需载入内存：挂载时读卷控制块，访问文件时读文件控制块，遍历路径时读目录节点；最终目录项与文件数据都落在磁盘块上。
- **易错/考点**：superblock / inode / dentry 的粒度分别是「每个文件系统 / 每个文件 / 每个目录项」，最易记错。

12.11 数据缓存（3.9 分钟）

- **主旨**：把文件数据块缓存到内存（含页缓存），以减少磁盘 I/O，并把文件缓存与虚拟内存页统一起来。
- **关键点**：
  1. 数据块按需读入内存：系统提供 read() 操作，读之前按读取范围确定所需的数据块。
  2. 数据块使用后被缓存，前提假设是数据将会再次被使用；写操作可能被缓存并延迟写回（delayed write）。
  3. 两种数据块缓存方式：普通缓冲区缓存（buffer cache）；页缓存（page cache），统一缓存文件数据与内存页。
  4. 页缓存依赖分页：需要一个页时才将其载入内存，虚拟地址空间中的一个页可被映射到二级存储中的本地文件。
  5. 文件数据块的页缓存把数据块映射成页，文件读写操作被转换成对内存的访问，可能导致缺页（page fault）和/或把页置脏。
  6. 遗留问题是页置换的取舍：回收时从进程还是从文件缓存中取。
- **易错/考点**：页缓存把文件 I/O 变成内存访问，因此缺页异常与脏页回写被纳入文件读写路径。

12.12 打开文件的数据结构（4.3 分钟）

- **主旨**：说明 open 之后内核为进程和系统分别维护哪些表，以及它们如何层层指向 VFS 中的文件对象。
- **关键点**：
  1. 打开文件描述：每个被打开的文件一个，包含文件状态信息、目录项、当前文件指针、文件操作设置等。
  2. 打开文件表分两级：进程打开文件表一个进程一个，另有系统级的打开文件表。
  3. 每个卷控制块中也保存一份打开文件列表，所以只要有文件被打开，对应卷就无法被卸载。
  4. 进程打开文件表的表项指向系统打开文件表的表项，后者再指向 VFS 树中的 file 节点，file 再指向 data block。
  5. 文件锁由部分操作系统和文件系统提供，用于调节对文件的访问，分强制与劝告两类。
  6. 强制锁根据锁的保持情况和需求直接拒绝访问；劝告锁只提供状态，由进程自己查找锁状态来决定怎么做。
- **易错/考点**：卷上有文件处于打开状态即不能卸载，这条最易被忽略。

12.13 文件分配（13.6 分钟）

- **主旨**：比较连续、链式、索引三种数据块分配方式在存储效率与访问速度上的取舍。
- **关键点**：

1. 两个评价指标：高效（存储利用，如外部碎片）与表现（访问速度）。
2. 连续分配用文件头记录起始块和长度，位置策略有最先匹配、最佳匹配；顺序与随机访问都高效，但存在碎片，且文件增长困难（预分配还是按需分配）。
3. 链式分配把文件按数据块链表存放，文件头含第一块与最后一块的指针；创建、增大、缩小都容易且没有碎片，但不能真正随机访问，可靠性差（破坏一个链节即丢失后续）。
4. 索引分配为每个文件建立一个名为索引数据块的非数据数据块，存放指向数据块的指针列表，文件头指向该索引块；支持直接访问，代价是小文件也要付索引开销，且大文件需另想办法。
5. 大文件的处理是链式索引块与多级索引块（一重/二重/三重间接）；UFS 文件头含 13 个指针，10 个直接，第 11、12、13 个分别指向一重、二重、三重间接数据块。
6. 该结构提高了文件大小限制阈值、扩展容易、小文件开销小，但访问大文件的数据块需要大量间接查询。

- **易错/考点**：索引块自身也占盘块，小文件时开销占比高；间接层数决定可支持的最大文件尺寸。

12.14 空闲空间列表（5.1 分钟）

- **主旨**：用位图或空闲块链/分组记录哪些磁盘块可用，并分析位图的规模与扫描代价。
- **关键点**：

1. 位图（bitmap）用每一位代表一个数据块：i = 0 表示块 i 空闲，反之表示已分配。
2. 位图使用简单但可能很大：1600GB 磁盘约 40M 个块，位图约占 5MB。
3. 若空闲块在磁盘中均匀分布，找到第一个「0」之前平均需扫描 n/r，n 为磁盘数据块总数，r 为空闲块数。
4. 位图必须保存在磁盘上，而内存与磁盘的副本可能不同，因此不能让内存中块 i 空闲、磁盘上块 i 却已分配。
5. 正确次序是先在磁盘上置 bit[i] = 1、再分配 block[i]、最后在内存中置 bit[i] = 1，避免崩溃后同一块被重复分配。
6. 另一类是链式列表与分组列表：空闲块串成链，或分组后由每组最后一块记录下一组首块（next group block），不必逐块遍历。

- **易错/考点**：位图更新顺序（先磁盘后内存）是隐蔽考点；分组链表只在组尾保存下一组指针。

12.15 多磁盘管理-RAID（14.3 分钟）

- **主旨**：用多个并行磁盘同时提升吞吐量与可靠性，并按冗余组织方式划分 RAID 级别。
- **关键点**：

1. 先区分两个概念：分区是磁盘驱动器上适合操作系统指定格式的划分；卷是一个拥有文件系统实例的可访问存储空间，通常驻留单个分区，也可跨分区甚至跨磁盘。

2. 多磁盘的两大收益：通过并行提高吞吐量；通过冗余提高可靠性和可用性。
3. RAID 0 把数据块切成子块分散存到独立磁盘（条带化），无冗余，靠更大的有效块尺寸提供更大磁盘带宽。
4. RAID 1 是镜像：向两个磁盘写入、可从任何一个读取，可靠性成倍增长，读取性能也成倍增加。
5. RAID 4 采用数据块级条带加专用奇偶校验盘，允许从任意一个故障磁盘中恢复，但校验盘成为写入瓶颈。
6. RAID 5 改为分布式奇偶校验，校验块在不同磁盘间轮转；另有 RAID 0+1（先条带后镜像）与 RAID 1+0（先镜像后条带）两种组合。

- **易错/考点**：RAID 0 无冗余，加盘只增带宽不增可靠性；RAID 4 与 RAID 5 的唯一实质差别是校验块集中还是分散。

12.16 磁盘调度（15.7 分钟）

- **主旨**：通过重排 I/O 请求顺序压缩寻道与旋转开销，并比较 FCFS、SSTF、SCAN、C-SCAN、C-LOOK、FSCAN 的差异。
- **关键点**：

1. 读写时磁头必须定位到期望的磁道并从期望扇区开始：寻道时间定位磁道，旋转延迟等待扇区转到磁头下，平均旋转延迟为旋转一周时间的一半。
2. 访问时间 $T_a = T_s + 1/(2r) + b/(rN)$，其中 T_s 为寻道时间、$1/r$ 为旋转一周的时间、b 为传输字节数、N 为每磁道字节数。
3. 调度目标：按顺序处理请求、公平对待所有进程；进程很多时性能接近随机调度（图示为锯齿状曲线）。
4. FCFS 按请求到达顺序服务（示例队列 98, 183, 37, 122, 14, 124, 65, 67，磁头起始于 53），公平但磁头来回大幅移动。
5. SSTF 每次选择从磁臂当前位置移动最少的请求，即最短寻道时间优先，代价是远端请求可能饥饿。
6. SCAN 沿一个方向移动并服务沿途请求，到达该方向最后磁道后反向，故称电梯算法（elevator algorithm）；C-SCAN 只在一个方向上扫描，到末端直接返回另一端重新扫描；C-LOOK 只到该方向最后一个请求处就立即反转。

- **易错/考点**：SCAN 与 C-SCAN 的回程是否服务请求是主要区分点；此外 SSTF/SCAN 会出现磁臂粘着（arm stickiness），N-step-SCAN 把请求队列分成若干长度为 N 的子队列依次扫描，FSCAN 只分两个队列并把扫描期间新到的请求推迟到下一轮。

附录 A：关于这批视频的几个提醒

1 组重复文件

`7.11 上下文切换` 与 `7.12 进程控制—创建进程` 两个文件大小相差不到 3KB、时长都是 4.9 分钟，抽帧比对后画面几乎完全一致（帧差 0.6/255，仅编码噪声）。**实际是同一段录像被存了两份**，7.12 的内容无法从这份文件得到。

所以：113 个文件里，有效内容是 112 段。如果你要补 7.12 的内容，得另找资源。

画面信息量偏低的几节

小节	情况
1.4 如何学习操作系统	全程只有几行方法论文字，没有技术内容
7.12 创建进程	与 7.11 重复，无法辨认
12.2 基本概念	只有 2 分钟，仅交代组织顺序，实质内容在后几节
实验课（第 0 章、1.x 的实验部分）	画面以 IDE 代码与终端为主，笔记只概括机制

附录 B：考前速览 40 条

把全书压成 40 句，每句对应一个必考点。考前 30 分钟扫一遍。

内存

1. 逻辑地址 → 物理地址：x86 走「段机制 + 页机制」两段转换，uCore 把段基址全设为 0，等于只做页转换。
2. 连续分配的三种放置策略：首次匹配（快）、最优匹配（留大块但产生小碎片）、最差匹配（留大块给后来者）。
3. 内碎片 = 分配给进程但用不完；外碎片 = 空闲但太小、谁也放不下。分页消除外碎片，只剩页内碎片。
4. 页表项关键位：P 存在、R/W 读写、U/S 用户态可否访问、A 已访问、D 已写（脏页）。
5. TLB（快表）缓存的是「页号→帧号」的转换结果，命中就省掉一次访存。
6. 多级页表用「按需分配页表页」换空间；反向页表让表的大小与物理帧数挂钩，与虚拟空间无关。
7. 缺页（page fault）属于异常而非中断，Error Code 中 P=0 表示页不在内存、P=1 表示保护违规。
8. EAT = (1−p)×访存时间 + p×缺页处理时间，p 是缺页率——这是所有虚存性能题的母公式。

置换算法

1. OPT 淘汰「未来最久不用」，无法实现，只用来给其他算法定下界。
2. FIFO 按进入顺序淘汰，会出现 Belady 异常（页框变多、缺页反而变多）。
3. LRU 用过去代替未来，符合局部性，但实现代价高（需要栈或时间戳）。
4. Clock 用循环链表 + 访问位，把 LRU 的开销降到接近 FIFO；二次机会法再加修改位，避免反复写回脏页。
5. 抖动（thrashing）的根因是页框少于工作集，靠调低多道程序度缓解，换算法没用。

进程与调度

1. 进程 = 程序 + 执行状态 + 资源；PCB 是它存在的唯一标志。
2. 三态模型：就绪 → 运行 →（等待）→ 阻塞 → 就绪；挂起是把整个进程换出到外存。
3. 线程 = 进程 − 共享资源；同一进程的线程共享地址空间，切换不用换页表。
4. fork() 创建、exec() 覆盖、wait() 回收、exit() 终止；子进程结束但父进程未 wait → 僵尸（zombie）。
5. 调度指标互相冲突：响应时间、周转时间、吞吐量、公平性、开销。没有全能算法。
6. SPN/SRT 平均等待时间最优但需要预知运行时间，且长作业会饥饿。
7. 轮转（RR）的时间片越小响应越快、上下文切换开销越大。
8. 优先级反转 = 低优先级占着锁、高优先级等锁、中优先级抢 CPU；解法是优先级继承或优先级天花板。

同步

1. 竞态条件的根源是「语句不是原子的」，加上并发调度顺序不确定。
2. 临界区四条件：互斥、前进（空闲让进）、有限等待、让权等待（不忙等）。
3. 关中断能保证单处理器互斥，但多处理器无效，且用户态无权关中断。
4. Peterson 算法用 flag + turn 两个变量解决两进程互斥；Bakery 算法推广到 n 个进程，按「取号」排队。
5. Test-and-Set / Exchange 是硬件原子指令，用它们可以构造自旋锁。
6. 信号量 P（wait）申请、V（signal）释放；**P 的顺序写反会导致死锁**。
7. 管程把互斥交给编译器，程序员只管条件变量；条件变量的 wait 必须放在 while 里而不是 if（防虚假唤醒）。
8. 生产者—消费者需要三个信号量：互斥 mutex、空位 empty、满位 full。
9. 读者—写者：读者优先会让写者饥饿，写者优先会让读者饥饿。
10. 哲学家进餐的死锁解法：最多允许 4 人同时拿叉、或按编号顺序拿、或只在左右都空闲时才拿（状态机法）。

死锁

1. 四个必要条件：互斥、占有并等待、非抢占、循环等待——破坏任一条即可预防。
2. 死锁避免 ≠ 预防：避免是运行时用「安全状态」判断这次分配会不会导致死锁。
3. 银行家算法：每次请求先试探性分配，再跑安全性检查，能找到安全序列才真正分配。
4. 死锁检测用资源分配图化简（单实例）或类似银行家的矩阵算法（多实例），检测到后靠终止进程或剥夺资源恢复。

文件系统

1. 文件 = 元数据（文件头/inode）+ 数据块；目录本质是一张「名字 → inode」的表。
2. 硬链接共享 inode，删除只是减引用计数；软链接是另一个文件，存的是路径，会悬空。
3. 三种块分配：连续（快、有外碎片、难扩展）、链式（易扩展、只能顺序访问）、索引（支持随机访问、索引块有开销）。
4. VFS 的作用：让 read/write 这些系统调用不必关心底下是 ext2 还是 FAT。
5. RAID 0 条带化提性能、RAID 1 镜像提可靠、RAID 5 分布式校验兼顾两者；磁盘调度 SSTF 会饥饿，SCAN/C-SCAN 不会。